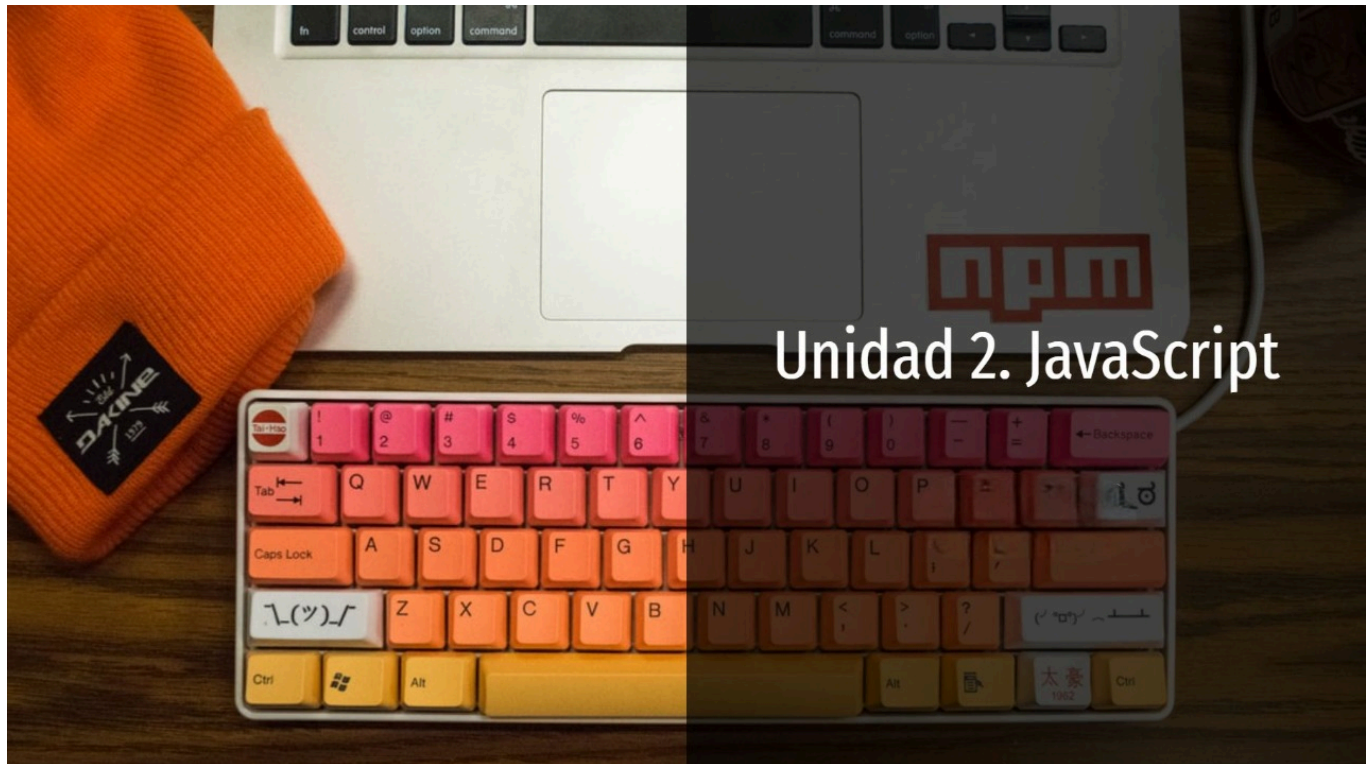


Unidad 2. Javascript

Unidad 2. Javascript



Índice de contenidos

1. Fundamentos del lenguaje JavaScript
 2. Funciones
 3. Objetos y prototipos
 4. Módulos
 5. Programación funcional
 6. JavaScript moderno
-

Objetivos de aprendizaje

1. Conocer las principales características del lenguaje JavaScript.

2. Dominar la sintaxis básica del lenguaje.
 3. Crear y utilizar funciones.
 4. Realizar operaciones con arrays.
 5. Entender el sistema de objetos y prototipos.
 6. Crear y utilizar objetos.
 7. Conocer el sistema de módulos.
 8. Conocer los fundamentos de la programación funcional.
 9. Aprender buenas prácticas de uso del lenguaje.
 10. Fomentar la curiosidad por ampliar los conocimientos del lenguaje.
-

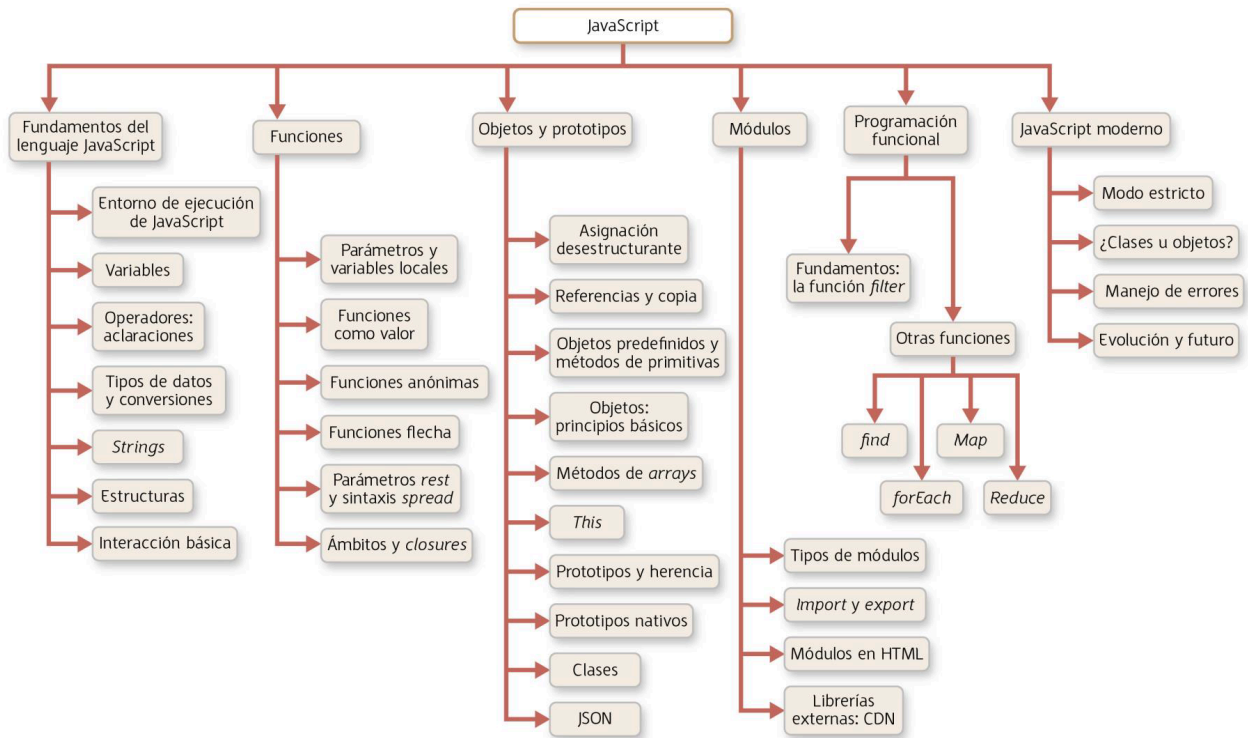
Descargas - Ejemplos para trabajar y analizar

A continuación, te ofrecemos una serie de archivos que debes descargar y tener disponibles en tu equipo para abordar el estudio de esta unidad didáctica.

- [Apartado 1. Ejemplos](#)
 - [Apartado 2. Ejemplos](#)
 - [Apartado 3. Ejemplos](#)
 - [Apartado 5. Ejemplos](#)
-

Síntesis de la unidad

Unidad 2. JavaScript



Síntesis de la unidad

Enlaces web

En esta sección encontrarás aquellos enlaces de interés necesarios para ampliar e investigar los contenidos de la unidad.

- **The Modern JavaScript Tutorial — Completo y moderno manual de JavaScript** <<https://javascript.info/>> *javascript.info*
- **JavaScript en MDN — Documentación de JavaScript de la Fundación Mozilla** <<https://developer.mozilla.org/es/docs/Web/JavaScript>> *developer.mozilla.org*
- **Fecha y Hora — Gestión de fecha y hora con el objeto Date() (The Modern JavaScript Tutorial)** <<https://es.javascript.info/date#creacion>> *es.javascript.info*
- **Date.prototype.toLocaleString() — Convertir fechas a cadenas en formato localizado (MDN)** <https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Date/toLocaleString> *developer.mozilla.org*
- **You Don't Know JavaScript — Libro sobre JavaScript (en inglés)** <<https://github.com/getify/You-Dont-Know-JS/tree/1st-ed>> *github.com*
- **JavaScript Factory Functions vs Constructor Functions vs Classes — Constructores, clases y factories** <<https://medium.com/javascript-scene/javascript->>

[factory-functions-vs-constructor-functions-vs-classes-2f22ceddf33e>](#)
medium.com

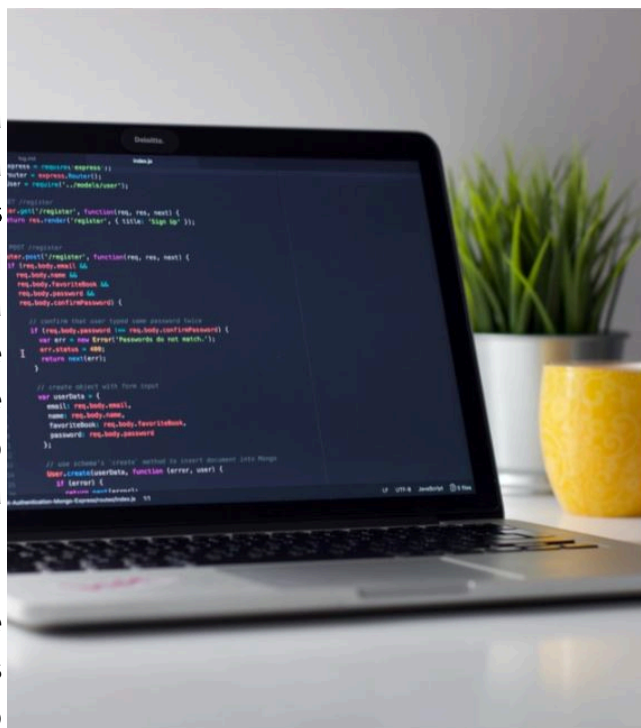
1. Fundamentos del lenguaje JavaScript

Presentación

El lenguaje JavaScript es la base del desarrollo web en entorno cliente. Es un lenguaje moderno que admite diferentes paradigmas de programación: **programación imperativa, procedimental, orientada a eventos, orientada a objetos y funcional**.

Pese a ser tratado en sus inicios como un lenguaje inferior, no apropiado para aplicaciones complejas, JavaScript se ha revelado como uno de los lenguajes **más utilizados** en el mundo del desarrollo. Su uso ha trascendido el navegador y ha pasado a ser usado también como lenguaje de desarrollo de aplicaciones **móviles, de escritorio** o de **servidor**. Gran culpa de ello ha tenido la aparición de la plataforma **NodeJS**.

Como todos los lenguajes de programación, JavaScript utiliza conceptos ya estudiados en el curso anterior, como variables, tipos de datos, funciones o estructuras de control. Sin embargo, también incorpora importantes **variaciones**, que modifican, amplían o bien simplifican dichos conceptos. Es importante familiarizarse con ellas para tener un mejor dominio del lenguaje.



Fundamentos del lenguaje JavaScript

En el módulo de **Lenguajes de marcas y sistemas de gestión de información** seguramente ya tuviste una introducción a JavaScript. En este apartado haremos hincapié en los aspectos más avanzados.

JavaScript es un lenguaje de una gran complejidad. No se pretende explicar absolutamente todas sus características, sino solo hacer hincapié en las más importantes. En las referencias, tienes acceso a varios recursos que ofrecen una visión más amplia del lenguaje.

1.1. Entorno de ejecución de JavaScript

JavaScript es un lenguaje que puede ser utilizado en el navegador, y ejecutado en el cliente y en un servidor (NodeJS). El código puede ir definido de las siguientes formas:

- En una página HTML. Debe ir entre etiquetas **<script>** (no es necesario indicar el atributo **type="script"**).
- En un archivo **.js** independiente. Dicho archivo puede ser ejecutado en NodeJS y cargado por una página HTML mediante **<script src="RUTA_ARCHIVO.js">**.

1.2. Variables

Los nombres de variables en JavaScript deben contener únicamente letras, dígitos numéricos o los caracteres `$` y `_`. El primer carácter no puede ser un dígito. Además, existe un conjunto de **palabras reservadas** que no pueden ser utilizadas como nombre de variables, como **function**, **let**, **for**, **if**, etcétera.

Existen tres maneras de declarar variables en JavaScript:

- Mediante **var**. La variable quedará definida al ámbito de la **función** en la que esté declarada. Si no está declarada dentro de una función, la variable quedará definida en el **objeto global**, del que hablaremos a lo largo de la unidad.
- Mediante **let**. Es la **forma recomendada**. La variable quedará definida en el ámbito del **bloque** en el que esté declarada. Un bloque es un trozo de código limitado por `{ }`. En el caso de estar definida fuera de cualquier bloque, quedará definida en el objeto global o en el módulo donde esté definida.
- Mediante **const**. Queda definida de la misma manera que **let**, **pero su valor no puede ser reasignado**.

Importante: JavaScript busca las variables definidas a través de lo que se denomina **ámbito léxico**, o *lexical scope*. Cuando se hace referencia a una variable en el interior de una función (en el caso de **var**) o un bloque (en el caso de **let**), JavaScript busca primero si existe una variable local o un parámetro con el identificador de la variable buscada: si lo encuentra, lo utiliza; si no, busca en el siguiente nivel de función (o bloque), y así sucesivamente, hasta que llega a la raíz, que es el objeto global (programa convencional) o el módulo (si el código está definido en un módulo).

En JavaScript moderno, se tiende a utilizar sobre todo **let** para definir variables, ya que las variables creadas de esta manera siguen reglas más parecidas a la mayoría de los lenguajes de programación.

Importante: Utiliza **let SIEMPRE** que vayas a utilizar una variable en un **BUCLE**, tanto si es la variable que vas a iterar como si es una variable auxiliar que vas a declarar de manera local en dicho bucle.

Ejemplo:

```
var a = 5; // Variable definida en el objeto global
var a = "hola"; // "var" permite redeclarar variables (aunque no es rec
let b = 6; // Variable definida en el objeto global
```



```

5 let b = "hola"; // Error: Uncaught SyntaxError: Identifier 'b' has already been declared
6
7
8 function funcion1() {
9
10     console.log(a); // Es correcto. Muestra "hola"
11     console.log(b); // Es correcto. Muestra 6.
12
13     var c = "hola"; // Variable definida en el ámbito de la funcion1
14
15     for (let d = 0; d < 5; d++) {
16         console.log(d); // d existe solo en este bloque
17     }
18     console.log(d); // Error: Uncaught ReferenceError: d is not defined
19
20     for (var d1 = 0; d1 < 5; d1++) {
21         console.log(d1); // d1 se crea en el ámbito de funcion1
22     }
23     console.log(d1); // Es correcto: "d1" está definida en el ámbito de funcion1
24
25     if (true) {
26         var e = 52;
27         let f = 60;
28     }
29
30     console.log(e); // Es correcto: "e" está definida en el ámbito de funcion1
31     console.log(f); // Error: "f" solo está definida en el bloque del "if"
32 }
33
34 console.log(c); // Uncaught ReferenceError: c is not defined (solo existe dentro de funcion1)
35
36 const g = 52;
37 g = "hola"; // Error: Uncaught TypeError: Assignment to constant variable.
38
39 const h = ["a", "b"];
40 h[0] = "d"; // Es correcto: "h" no se reasigna, sino que solo se cambia su primer elemento

```

Importante: Siempre **declara** una variable mediante **let**, **const** o **var** antes de utilizarla. Si no lo haces, JavaScript creará una variable **automáticamente** en el **objeto global**, con consecuencias inesperadas. Otra solución consiste en utilizar el **modo estricto**, poniendo **"use strict";** al principio de tu archivo JavaScript. Esto se trata de asignar un valor a una variable no declarada.

1.3. Operadores: Aclaraciones

El operador suma, +, se utiliza también para **concatenar cadenas de caracteres**. Además, si uno de los operandos es una cadena, se realiza la **conversión a cadena** del otro operando:

```
1 | console.log('1' + 2); // "12"
2 | console.log(2 + '1'); // "21"
```

Así, para utilizar el operador **suma** hay que **convertir las cadenas a números** previamente. Veremos cómo hacerlo en el siguiente punto.

Esto no ocurre con el **resto de operadores**, que realizan la **conversión a número**:

```
1 | console.log(6 - '2'); // 4
2 | console.log('4' / '2'); // 2
```

Los operadores también permiten realizar la operación de realizar **modificaciones sobre la misma variable**. Así:

```
1 | let a = 5;
2 | a += 5; // Es lo mismo que (a = a + 5)
3 | a /= 3; // Es lo mismo que (a = a / 3)
```

1.4. Tipos de datos y conversiones

Los valores en JavaScript son de un tipo de datos siempre:

- **Primitivos:**
 - **number**. Para números, enteros o decimales.
 - **bigint**. Para números muy grandes.
 - **string**. Para cadenas de caracteres.
 - **boolean**. Verdadero (**true**) o falso (**false**).
 - **null**. Valor nulo.
 - **undefined**. Para variables declaradas no inicializadas (que no han recibido ningún valor).
 - **symbol**. Es un tipo de datos para crear **identificadores únicos e inmutables**, que se suelen utilizar para definir claves de objetos.
- **object**. Para **objetos** (incluidos *arrays* y funciones).

Dado que JavaScript es un lenguaje de tipado **dinámico**, las variables pueden albergar cualquier tipo de datos. Esto puede crear problemas en tiempo de ejecución, ya que al efectuar operaciones es posible que se produzcan **conversiones de tipos**. Consideremos el siguiente ejemplo:

```
1 let edad = prompt("Dime tu edad");
2 let edadMasDiez = edad + 10;
3 console.log(edadMasDiez);
4
5 // El usuario introduce 40. Pero "prompt" siempre devuelve un string, a
6 // Imprime "4010" (en lugar de 50)
7 // Se ha producido la conversión de tipo de 10 a "10",
8 // ya que el operador "+" se ha comportado como "concatenar" en lugar d
```

Las funciones y expresiones **matemáticas** realizan conversiones a **número** (con la excepción del operador **+**, que se convierte en el operador «concatenar», como acabamos de ver). Los casos especiales son:

Valor	Conversión
undefined	NaN
null	0
true / false	1/0

string	Si es una cadena vacía, 0 ; si se corresponde con un número, el número en cuestión; si no corresponde a un número válido, NaN .
---------------	---

El valor **NaN** es un valor numérico que significa ***Not a Number***.

Para **convertir a número** podemos utilizar **varios métodos**:

```

1  parseInt("3");           // 3
2  parseFloat("3.155");    // 3.155
3  Number("3");           // 3
4  +"3";                  // 3 (Operador unario)
5
6  console.log(+ "3" + 4);  // 7
7
8  // Ejemplo anterior ejecutado de manera correcta
9  let edad = prompt("Dime tu edad"); // El usuario introduce 40.
10 let edadMasDiez = Number(edad) + 10; // Convertimos "edad" a número
11 console.log(edadMasDiez);          // 50

```

Es posible **comprobar si una cadena de texto no corresponde a un número** mediante la función **isNaN** (*is not a number*):

```

1  isNaN("5rt");           //true (no es un número válido)
2  isNaN("541");           // false
3  isNaN("285.32");        // false
4  isNaN("2e16");          // false

```

Las conversiones a **string** se realizan automáticamente. Por ejemplo, la función **alert** puede tomar como parámetro un número que convierte automáticamente a **string**. También se puede realizar la conversión explícita mediante **String(valor)**.

Las conversiones a **boolean** siguen las siguientes reglas:

Valor	Conversión a boolean
0, null, undefined, NaN, ""	false
Otro valor (incluida la cadena "0")	true

Los operadores de comparación de igualdad o desigualdad tienen dos versiones:

- con comprobación de tipo (===, !==) o
- sin comprobación de tipo (==, !=, conversión de tipo automática).

```
1 | let valor;  
2 | valor = (0 == false); // true  
3 | valor = ('' == false); // true  
4 | valor = (0 === false); // false (distinto tipo)  
5 | valor = ('' === false); // false (distinto tipo)
```

Importante: `null` y `undefined` son iguales `==` (igualdad no estricta) entre sí y distintos a cualquier otro valor.

Para terminar, una **recomendación**: Si una variable puede almacenar `null` o `undefined`, compruébalo por separado mediante `==` (o `===`, si quieres diferenciar entre `null` y `undefined`) antes de realizar cualquier otra comparación.

1.5. Strings

Las cadenas de texto o **strings** pueden definirse mediante **comillas dobles** (") o **sencillas** ('). Si se desea utilizar comillas dentro de una cadena, habrá que utilizar un entrecomillado distinto. Por ejemplo:

```
1 | let a = "Texto con 'comillas' dentro";
```

Existe una tercera manera de definir una cadena de texto: mediante **acento grave** (`). Este modo tiene las siguientes ventajas:

- Permite crear cadenas de texto de **varias líneas**.
- Permite **incorporar expresiones** utilizando la estructura `${ }` dentro de la propia cadena. Esto evita tener que utilizar el operador de concatenar `+` y pensar en los espacios en blanco que se deben dejar para formatear adecuadamente textos que incorporen expresiones o datos almacenados en variables.

Ejemplo:

```
1 | let cadena1 = `Cadena
2 | formada
3 | por
4 | varias
5 | líneas`;
6 |
7 | let nombre = "Pedro";
8 | let cadena2 = `Hola, ${nombre}.`;
9 | // Esta última línea es equivalente a:
10 | // let cadena2 = "Hola, " + nombre;
11 | // Nótese el espacio en blanco después de la coma
12 |
13 | let cadena3 = `3 por 4 es igual a ${3 * 4}`;
14 | // 3 por 4 es igual a 12
```

Importante: El acento grave en el teclado español se muestra pulsando la tecla correspondiente del teclado (a la derecha de la «P») y pulsando la barra espaciadora a continuación.

1.6. Estructuras de Control y Bucles

JavaScript dispone de las estructuras clásicas de los lenguajes de programación: **condicionales** (**if**, **switch**) y **bucles** (**for**, **while**). Dichas estructuras han sido estudiadas en el primer curso del ciclo, por lo que se supone que son conocidas. Por ello, nos centraremos en estructuras algo más avanzadas.

La primera que veremos es el **operador ternario**, **?**. Este operador permite evaluar una condición de manera más concisa. Se utiliza para **asignar un valor a una variable en función de una condición**. Por ejemplo:

```
let edad = prompt( '¿Cuántos años tienes?' );
let mensaje;

if( edad > 30 ) mensaje = "Tienes más de 30 años.";
else           mensaje = "Tienes menos de 30 años.";

alert( mensaje );
```

El condicional puede reescribirse como:

```
let edad = prompt( '¿Cuántos años tienes?' );

let mensaje = (edad > 30) ? "Tienes más de 30 años." : "Tienes menos de 30 años.";

alert( mensaje );
```

La segunda estructura de interés es el bucle **for of**, que se utiliza para iterar sobre arrays:

```
let colores = [ "azul", "rojo", "blanco" ];

// Bucle "for" tradicional
for( let i = 0; i < colores.length; i++ )
    console.log( colores[ i ] );

// azul
// rojo
```

```
// blanco

// Bucle "for of"
for( let color of colores )
    // La variable "color" almacena el elemento del array que se esté iterando
    // Equivalencia: color = colores[i]
    console.log( color );

// azul
// rojo
// blanco
```

Importante: Existe otra estructura parecida, **for in**. La diferencia es que **for in** se utiliza para iterar sobre las propiedades de un objeto. Como en JavaScript los *arrays* son objetos, también funcionará, aunque no es recomendable usarla, ya que es más lenta y accede también a otras propiedades que no son elementos del *array*.

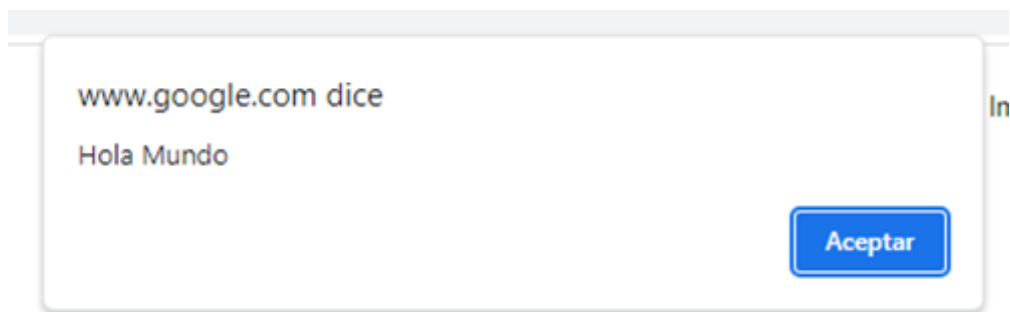
1.7. Interacción Básica

El navegador puede interactuar de forma básica con el usuario mediante tres funciones: `alert()`, `prompt()` y `confirm()`.

Importante: Estas funciones no están disponibles en el servidor (**NodeJS**) porque son parte de la API del navegador (dentro del objeto **window**).

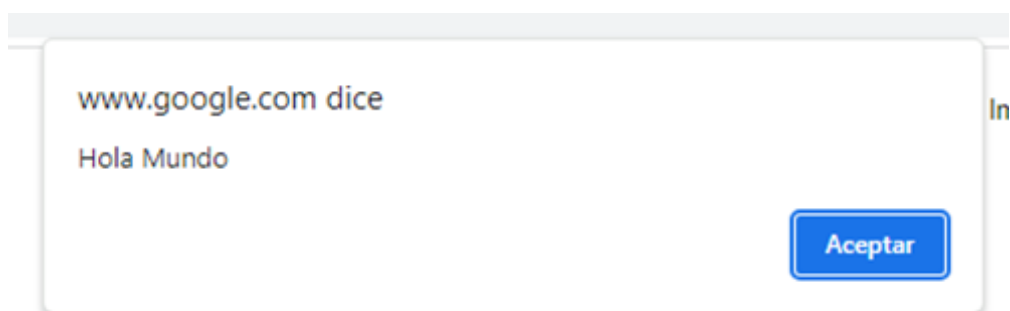
La función `alert()` muestra, en un cuadro de diálogo, el mensaje de texto pasado como argumento a la función. El cuadro de diálogo incluye el botón "Aceptar".

```
alert( "Hola Mundo" );
```



La función `alert()` muestra, en un cuadro de diálogo, el mensaje de texto pasado como argumento a la función. El cuadro de diálogo incluye el botón "Aceptar".

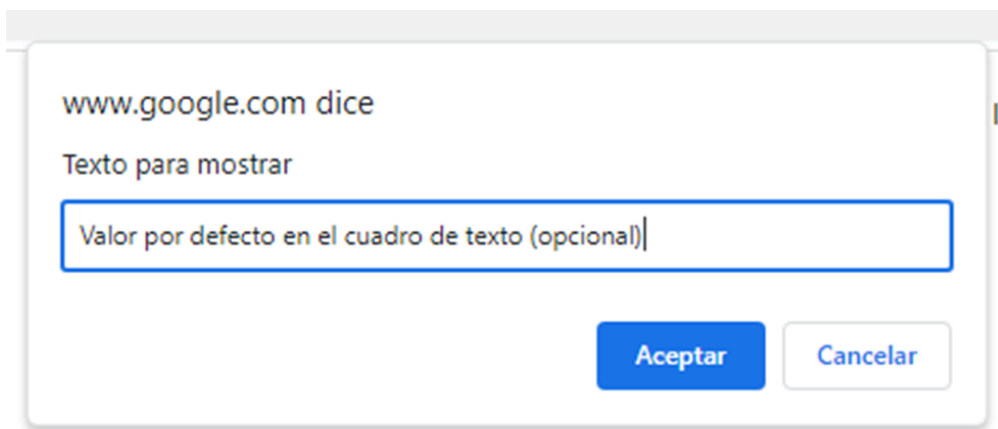
```
alert( "Hola Mundo" );
```



La función **prompt()** muestra un cuadro de diálogo con un campo de texto para que el usuario introduzca un valor en el navegador, junto con dos botones, "Aceptar" y "Cancelar".

La función tiene dos parámetros; el primero es el texto a mostrar en el cuadro de diálogo y el segundo, que es opcional, es el texto por defecto a mostrar en el campo de texto.

```
let respuesta = prompt( "Texto para mostrar", "Valor por defecto en el cuadro de texto" );  
  
// Importante: La variable "respuesta" almacena el texto escrito por el usuario
```



Importante

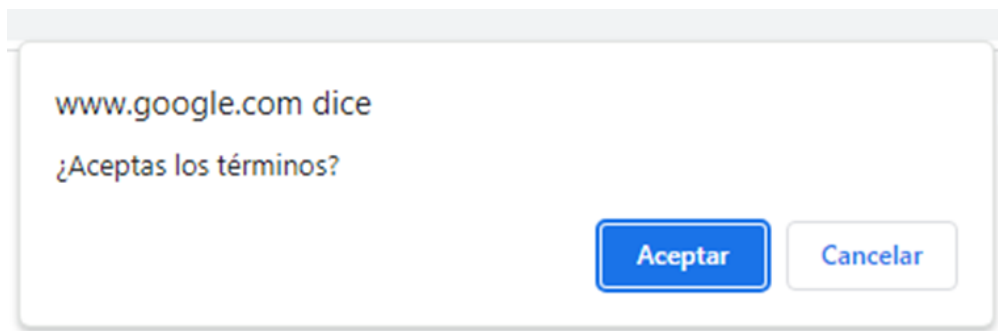
prompt devuelve un **string** si el usuario escribe algo, y **null** si el usuario cancela la acción.



¿Qué muestra la función **confirm()** en el navegador?

La función **confirm** muestra un cuadro de diálogo con un mensaje de texto junto con dos botones, "Aceptar" y "Cancelar". La función devuelve **true** si el usuario pulsa "Aceptar" y **false** si pulsa "Cancelar".

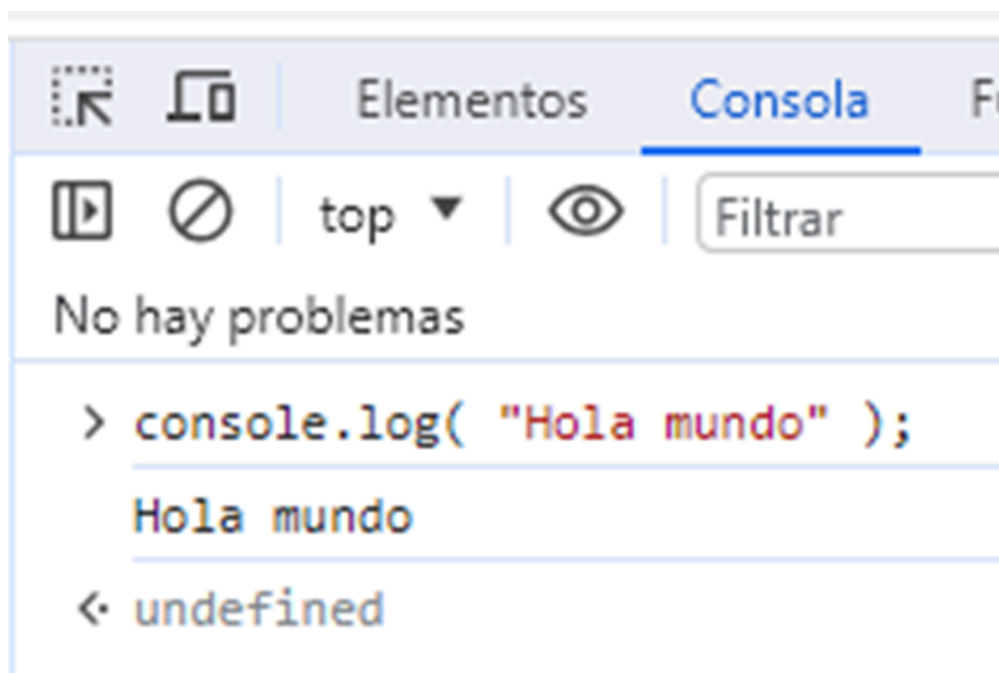
```
let acepta = confirm( "¿Aceptas los términos?" );
```



¿Cómo se escribe un mensaje de texto en la consola?

La función `console.log( "Mensaje a mostrar" );` permite escribir un mensaje de texto en la consola. Esta función está disponible tanto en el navegador como en NodeJS.

```
console.log( "Hola mundo" );
```



Test 1

1. Fundamentos del lenguaje JavaScript

1. Indica qué nombre de variable es válido en JavaScript.

- `$_var1`
- `1_$var`
- `1var`
- `var-1`

2. Indica la respuesta correcta.

- `let` define variables a nivel de función.
- `let` define variables a nivel de bloque.
- `var` define variables a nivel de bloque.
- `const` define variables a nivel de función.

3. Indica la respuesta correcta.

- `("5" + 4) == 9`
- `("5" + "4") == 9`
- `("5" + "4") == 9`
- `("5" + 4) == 54`

4. Indica cuáles de las siguientes afirmaciones son verdaderas (puede haber más de una respuesta válida).

- `0 == false`
- `0 === false`
- `'' == false`
- `'' === false`

5. Para iterar los elementos de un array, podemos utilizar...

- `for...in`
 - `for...of`
 - `for...each`
 - `for...while`
-

Actividad 2.1

Interacción básica y cadenas de texto

Realiza un programa JavaScript que se ejecute en una página HTML. Dicho programa deberá efectuar las siguientes acciones:

1. Pedir al usuario que introduzca un primer número.
 2. Pedir al usuario que introduzca un segundo número.
 3. Mostrar en pantalla un cuadro de alerta con un texto formado por dos líneas:
 - En la primera deberá mostrarse el texto: **"El resultado de sumar X y Y es Z."**, en el que **X, Y y Z** son los valores correspondientes.
 - En la segunda línea, deberá mostrarse el texto: **"Gracias por usar este programa"**.
-

Actividad 2.2

Ámbito de variables

Analiza los siguientes fragmentos de código suponiendo que se ejecutan de manera independiente en el objeto global y responde a las siguientes preguntas, para cada uno de ellos:

1. ¿Se genera algún tipo de error?
2. ¿Qué resultado mostrará cada uno de ellos en la consola?

```
//fragmento i
for (var i = 0; i < 5; i++) {
    console.log(i);
}
console.log(i);

//fragmento j
for (let j = 0; j < 5; j++) {
    console.log(j);
}
console.log(j);
```

Claves de resolución: Fíjate en cómo se declaran las variables de iteración de cada bucle. ¿Cuál es el ámbito en el que quedan definidas?

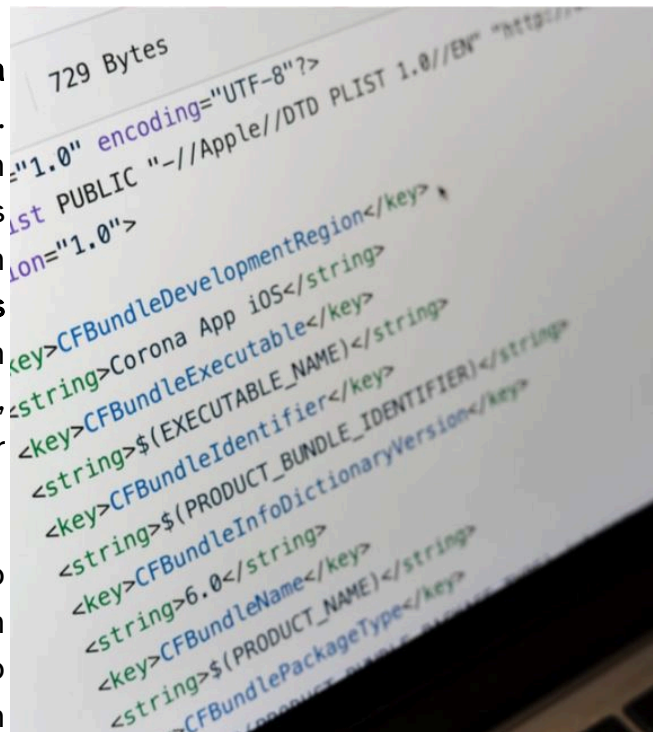
2. Funciones

Presentación

Las funciones constituyen uno de los pilares básicos de JavaScript. Se utilizan, como en cualquier lenguaje de programación, para **encapsular un trozo de código que realiza una tarea concreta**. Su utilización favorece la **organización** y la **reutilización** del código.

Las funciones en JavaScript **afectan a la visibilidad** de las variables del programa. Dependiendo de dónde y cómo se declaren las variables, podrán o no ser utilizadas dentro y fuera de las funciones. Existen también **distintos tipos, funciones convencionales y funciones flecha**, con comportamientos diferentes. Es necesario, por tanto, conocer estas reglas para poder hacer un uso adecuado del lenguaje.

JavaScript trata las funciones como cualquier otro **valor**, por lo que pueden **asignarse** a variables, pasarse como **parámetro** a otras funciones o definirse en el momento de utilizarse como **funciones anónimas**. Este estilo de programación se denomina **programación funcional**, uno de los paradigmas admitidos por JavaScript, junto con la programación orientada a objetos, la programación orientada a eventos y la programación procedimental.



2.1. Parámetros y Variables Locales

Las funciones en JavaScript pueden ser definidas de manera tradicional mediante **function**. Como en la mayoría de los lenguajes de programación, las funciones admiten **parámetros** y permiten la declaración de **variables locales** en su interior. JavaScript permite también el acceso a variables definidas fuera de la función: son las **variables globales**. Esta característica es consecuencia del **ámbito léxico**, visto en la sección de variables del primer apartado de la unidad.



Importante

En general, es buena idea minimizar el uso de variables globales, ya que pueden crear problemas de mantenimiento del código.

Las variables globales son accesibles, a no ser que se defina una variable local con el mismo nombre, en cuyo caso la variable local es la que será referenciada (se dice que la variable local **enmascara** la variable global).



Ejemplo 1

```
let nombre1 = "Pedro"; // Variable global
let nombre2 = "Pablo"; // Variable global

function nombres() {
  let nombre3 = "Juan"; // Variable local
  let nombre2 = "Jorge"; // Variable local que enmascara la variable global

  console.log(nombre1); // Pedro
  console.log(nombre2); // Jorge
  console.log(nombre3); // Juan
}

console.log(nombre1); // Pedro
```



```
console.log(nombre2); // Pablo  
console.log(nombre3); // undefined
```

Los parámetros de las funciones pueden incorporar **valores por defecto**:

```
function pintar(color="negro") {  
    console.log(`Pintando con color ${color}`);  
}  
  
pintar(); // Pintando con color negro  
pintar("azul"); // Pintando con color azul
```

2.2. Funciones como Valor

Las funciones en JavaScript son objetos, y, como tales, **pueden ser almacenadas en variables**:

```
function sumar(a, b) {  
    return a + b;  
}  
  
let suma1 = sumar(4,5); // suma1 = 9  
let funcSuma = sumar; // funcSuma almacena la función, no el resultado de su  
funcSuma(3,4); // 7  
let suma2 = funcSuma(3,6); // suma2 = 9
```

También pueden ser **pasadas como parámetros a otras funciones**:

```
function sumar(a, b) {  
    return a + b;  
}  
  
function restar(a, b) {  
    return a - b;  
}  
  
function operar(operacion, a, b) {  
    return operacion(a,b);  
}  
  
let v1 = operar(sumar, 4, 5); // v1 = 9  
let v2 = operar(restar, 4, 5); // v2 = -1
```

La aplicación más útil de esta característica es el uso de ***callbacks***, que son funciones que se ejecutan cuando se completa una tarea.



Ejemplo 2

```
function confirmar( pregunta, si, no ) {  
    if( confirm( pregunta ) )    si();    // Se ejecuta la función de callback  
    else                        no();    // Se ejecuta la función de callback  
}  
  
function aceptar() {  
    console.log( "Has respondido que sí" );  
}  
  
function cancelar() {  
    console.log( "Has respondido que no" );  
}  
  
// Se pasan las funciones 'aceptar' y 'cancelar' como parámetros  
// IMPORTANTE: se pasan los nombres de las funciones,  
// no el resultado de su ejecución  
// (Es decir, se pasa 'aceptar', no 'aceptar()'  
confirmar( "¿Sabes qué es un callback?", aceptar, cancelar );  
  
// Si queremos hacer otro tipo de procesamiento podemos pasar otro callback  
function aceptar2() {  
    alert( "Has respondido que sí" );  
}  
  
confirmar( "¿Sabes qué es un callback?", aceptar2, cancelar );
```

En el ejemplo, el parámetro `si()` de la función `confirmar()` almacena la función que se pasa como parámetro; en este caso, `aceptar` (sin paréntesis). Si el usuario responde afirmativamente en el cuadro de diálogo, se ejecuta dicha función mediante `si()` [como `si = aceptar`, por lo que `si()` equivaldrá a `aceptar()`]. El uso de *callbacks* permite **separar** el código de la **función principal** de la **acción que se ejecutará** al completar la tarea.



Importante

Es muy importante **comprender cuándo hay que utilizar () y cuándo no** al utilizar *callbacks*.

Estudiaremos los *callbacks* con más detalle en otra unidad.

2.3. Funciones Anónimas

Las funciones anónimas son funciones que no tienen nombre definido. En otros lenguajes reciben el nombre de **funciones lambda**. Por ejemplo:

```
let sumar = function( a, b ) {  
    return a + b;  
}  
  
let v1 = sumar( 4, 5 ); // 9
```

En este caso, **sumar** es una variable que almacena una función anónima. También podríamos utilizar una función con nombre:

```
let sumar = function sumar( a, b ) {  
    return a + b;  
}  
  
let v1 = sumar( 4, 5 ); // 9
```

Ambos casos son muy parecidos. La principal diferencia radica en la **depuración de programa**: en el primer caso, la función no tendrá nombre; en el segundo caso, sí que aparecerá identificada con un nombre, por lo que su identificación será más sencilla.

Es muy común utilizar **funciones anónimas** en los **callbacks**. Así, el ejemplo anterior de función **confirmar** quedaría de la siguiente manera:

```
function confirmar( pregunta, si, no ) {  
    if( confirm( pregunta ) ) si();  
    else no();  
}  
  
// En este caso, las funciones de callback se crean in situ como funciones anónimas  
confirmar( "¿Sabes qué es un callback?",  
    function() { console.log( "Has respondido que sí" ); },  
    function() { console.log( "Has respondido que no" ); } );
```



Importante

Ten especial cuidado con el cierre de **paréntesis** y **llaves** al utilizar *callbacks* y funciones anónimas. Es muy común encontrarte con el patrón `} )` como cierre.

2.4. Funciones Flecha

Las funciones flecha permiten definir funciones mediante la siguiente sintaxis:

```
let func = (par1, par2,... parN) => expresión;
```

Así, las siguientes funciones:

```
function sumar(a, b) {  
    return a + b;  
}  
  
function restar(a, b) {  
    console.log("Restando:");  
    return a - b;  
}
```

se podrían expresar como funciones flecha de la siguiente manera:

```
// El cuerpo de la función solo tiene una línea. Se pueden  
// omitir las llaves y el 'return'  
let sumar = (a, b) => a + b;  
  
// El cuerpo de la función tiene varias líneas. Se deben  
// incluir las llaves y el 'return'  
let restar = (a, b) => {  
    console.log("Restando");  
    return a - b;  
}
```



Importante

Estas funciones **no son simplemente versiones simplificadas** de las funciones convencionales, sino que tienen una serie de **características muy importantes** que hay que conocer. Algunas de ellas las comprenderemos más adelante, cuando tratemos los objetos. Dichas características son las siguientes:

- Son siempre **funciones anónimas**.
 - No admiten los métodos **call**, **apply** y **bind**.
 - No tienen su propio **this**, por lo que no son adecuadas para métodos de objetos.
 - No pueden utilizarse como constructores con **new**.
-



developer.mozilla.org: Funciones Flecha

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Functions/Arrow_functions

<https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Functions/Arrow_functions>

2.5. Parámetros "Rest" y Sintaxis "Spread"

En ocasiones, es interesante pasar a una función un **número variable de argumentos**. Por ejemplo, en una función que se encargue de calcular una media aritmética de un conjunto de números. En este caso, es útil el operador `...` delante del parámetro que vaya a recibir dichos datos, que pasarán a estar disponibles en un **array**.



Ejemplo 3

```
function calcularMedia( ...numeros ) {  
    // El parámetro 'numeros' es un array que contiene el listado  
    // de los parámetros pasados en la llamada a la función  
  
    let suma = 0;  
    let cantidad = numeros.length;  
  
    if( cantidad == 0 ) return 0;  
  
    for( let num of numeros )  
        suma += num;  
  
    return suma / cantidad;  
}  
  
let media1 = calcularMedia( 4, 6, 5 );           // 5  
let media2 = calcularMedia( 20, 40, 80, 60 );    // 50  
let media3 = calcularMedia( 3, 16, 22, 98, 16, 40 ); // 32.5
```



Importante

Si la función tiene muchos parámetros, solo uno de ellos, **el último**, puede utilizarse como parámetro *rest*.

Por otro lado, puede ser interesante realizar la **operación inversa**: queremos **convertir un array a un conjunto de parámetros**. Para ello se utiliza de nuevo el operador `...` delante del *array* que hay que convertir. Por ejemplo:

```
function sumar(a, b) {  
    return a + b;  
}  
  
let numeros1 = [4, 7];  
sumar(...numeros1); // 11
```

Uno de los usos más interesantes de este operador es la **copia de arrays** o la creación de **arrays nuevos** a partir de *arrays* existentes y elementos nuevos:

```
let colores1 = [ "azul", "rojo", "verde" ];  
let colores2 = colores1; // NO ES UNA COPIA, sino que hay un único array ref  
let colores3 = [ ...colores1 ]; // colores3 almacena un ARRAY NUEVO, distinto  
let colores4 = [ "cian", ...colores1, "rosa" ]; // colores4 almacena un ARRAY  
  
// Modifico el primer array  
colores2[ 0 ] = "amarillo";  
  
console.log( colores1[ 0 ] ); // amarillo  
console.log( colores2[ 0 ] ); // amarillo (mismo array)  
console.log( colores3[ 0 ] ); // azul (array distinto)  
console.log( colores4.length ); // 5  
console.log( colores4 ); // [ 'cian', 'azul', 'rojo', 'verde', 'rosa' ]
```



Importante

Estos conceptos tienen cierta complejidad, por lo que no se abordarán en profundidad. En las referencias puedes encontrar explicaciones más detalladas sobre el tema.



developer.mozilla.org: Parámetros Rest

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Functions/rest_parameters

<https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Functions/rest_parameters>



developer.mozilla.org: Sintaxis Spread

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Operators/Spread_syntax

<https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Operators/Spread_syntax>



freecodecamp.org: Operadores Rest y Spread de JavaScript: ¿Cuál es la diferencia?

<https://www.freecodecamp.org/espanol/news/operadores-rest-spread-de-javascript-cual-es-la-diferencia/> <<https://www.freecodecamp.org/espanol/news/operadores-rest-spread-de-javascript-cual-es-la-diferencia/>>

2.6. Ámbitos y Closures

Tal como hemos adelantado en el primer apartado de la unidad, el **ámbito léxico** hace referencia a la **zona de influencia** de un determinado trozo de código. Así, podemos definir el **ámbito interno de una función** como el **cuerpo de esta**; definimos el **ámbito externo de una función** como el **código que hay fuera de su cuerpo**.

Una **closure** es una **función que guarda referencias al ámbito exterior** aun después de haberse ejecutado el código de dicho ámbito. Es muy habitual encontrarlas en **funciones que devuelven funciones**. Consideremos el siguiente caso clásico:

```
function crearContador() {  
    let cuenta = 0;  
  
    // Función closure  
    return function() {  
        // Referencia a la variable "cuenta" del ámbito exterior a la función  
        return cuenta++;  
    };  
}  
  
let contador1 = crearContador();  
console.log( contador1() ); // 0  
console.log( contador1() ); // 1  
console.log( contador1() ); // 2  
  
let contador2 = crearContador();  
console.log( contador2() ); // 0  
console.log( contador2() ); // 1
```

En dicho ejemplo podemos ver una función, **crearContador**, que devuelve una función cuando se ejecuta. Por tanto, las variables **contador1** y **contador2** hacen referencia a su vez a funciones, las cuales son **closures**, ya que guardan una referencia a la variable **cuenta**. Es importante ver que tanto **contador1** como **contador2** guardan una referencia a **su propia versión de cuenta**.

Lo difícil de entender de las **closures** es que **el estado exterior no se elimina tras su ejecución**, sino que sigue referenciado y por tanto sigue estando presente. En nuestro ejemplo, la variable **cuenta** parece que debería desaparecer una vez acaba la ejecución

de **crearContador**: sin embargo, como dicha variable sigue referenciada por la función *closure*, sigue existiendo.



Importante

Las *closures* tienen mucha importancia en la **programación asíncrona**, ya que en ella se utilizan funciones que no se ejecutan inmediatamente, sino después de completar un proceso (petición de red, acceso a disco, tiempo de espera, etcétera).

También se utilizan como **alternativa a los objetos con propiedades privadas** (en el ejemplo anterior, la variable **cuenta** no es accesible desde el programa principal).



developer.mozilla.org: Closures

<https://developer.mozilla.org/es/docs/Web/JavaScript/Closures>

<<https://developer.mozilla.org/es/docs/Web/JavaScript/Closures>>

Test 2

2. Funciones

1. En JavaScript se puede asignar una función a una variable.

- Verdadero
- Falso

2. Es posible acceder a una variable definida fuera de una función mediante `let` desde dentro de dicha función.

- Verdadero
- Falso

3. Una función flecha...

- Se comporta exactamente igual que una función convencional.
- No puede ser anónima.
- No define su propio `this`.
- Solo puede tener un parámetro.

4. Dada la definición de función `function miFuncion(...p)`, ¿cuáles de las siguientes llamadas no es válida? (puede haber más de una respuesta válida).

- `miFuncion("a", "b", "c");`
- `miFuncion("a");`
- `miFuncion();`
- `miFuncion(["a", "b", "c"]);`

5. Dada esta definición de función, `function miFuncion(...p)`, ¿qué almacena el parámetro `p` si se le llama mediante `miFuncion(["a", "b", "c"]);`?

- `[['a', 'b', 'c']]`
 - `['a', 'b', 'c']`
 - `"abc"`
 - `"a,b,c"`
-

Actividad 2.3

Funciones con Parámetros Variables

Crea una función JavaScript que admita un conjunto variable de números. La función deberá devolver la media aritmética de los números introducidos. Implementa dicha función sin hacer uso de otras funciones (ya que esto se hará en la Actividad 2.10). A continuación, se muestran ejemplos de llamada a la función:

```
media(); // 0
media(2, 4); // 3
media(1); // 1
media(3, 5, 7, 14); // 7.25
```

Actividad 2.4

Funciones flecha

Reescribe las siguientes funciones con **sintaxis de función flecha** de la manera más simplificada posible. No debes modificar ninguna línea de su código: debes respetar el número de líneas que hay en su interior.

```
function funcion1(a, b) {  
    return (a + b) / 2;  
}  
  
function funcion2(a) {  
    let b = 5;  
    return a + b;  
}  
  
function funcion3(a, b) {  
    let c = 10;  
    return a + b + c;  
}
```

Claves de resolución: Las funciones flecha disponen de sintaxis simplificada en función del número de parámetros y el número de líneas de código que hay en su cuerpo.

3. Objetos y prototipos

Presentación

La programación orientada a objetos es otro de los paradigmas admitidos por JavaScript. A diferencia de otros lenguajes de programación, como Java o C#, JavaScript **no utiliza clases**, sino que **utiliza exclusivamente objetos**. Su filosofía es, por tanto, muy distinta de la de esos otros lenguajes.

Una de las características más difíciles de comprender del sistema de objetos de JavaScript es el uso de **this**. El mecanismo de funcionamiento de **this** es **dinámico**, y por ello puede hacer referencia a objetos diferentes **en función de cómo se llame a las funciones** que lo utilizan. El uso de **this** no depende, pues, de la definición de la función, sino de cómo se realiza su llamada.

JavaScript también define un mecanismo para enlazar y establecer relaciones entre objetos. Este concepto está relacionado con la **herencia** en los lenguajes que utilizan clases. Sin embargo, el mecanismo que utiliza JavaScript se basa en la **delegación de prototipos**. La correcta comprensión de esta característica es imprescindible para poder tener un buen dominio del lenguaje.



3.1. Objetos: Principios Básicos

Los **objetos** son el tipo de datos complejo disponible en JavaScript. Un objeto está formado por **colecciones de datos** definidas como **propiedades**.

Importante: Es muy importante tener claro que JavaScript tiene objetos, pero **no tiene clases**, por lo que es diferente de otros lenguajes como Java. Sí que existe la posibilidad de definir objetos mediante la palabra reservada **class**, pero no tiene nada que ver con las clases de lenguajes como Java.

La manera más sencilla de crear un objeto en JavaScript es mediante un literal:

```
let coche = {
  color: "rojo",
  marca: "seat",
  arrancar: function() {
    console.log("Arrancando");
  }
}

console.log( coche.color ); // rojo
console.log( coche.marca ); // seat
coche.arrancar();           // Arrancando
```

Se pueden **añadir propiedades** a un objeto directamente. Existen dos modos de hacerlo:

```
coche.modelo = "ibiza";
coche[ "combustible" ] = "diesel";
```

Se pueden iterar las propiedades de un objeto con el operador **for in**. Así, en el ejemplo anterior:

```
for( let prop in coche ) {
  console.log( prop );
}

// color
// marca
```

```
// arrancar  
// modelo  
// combustible
```

Si se accede a una propiedad de un objeto que no existe, el código **no genera error**, sino que devuelve **undefined**. Es posible detectar si una propiedad está definida en un objeto mediante el operador **in**:

```
let persona = {  
  edad: 20,  
  nombre: "David"  
}  
  
console.log( "edad" in persona );    // true  
console.log( "apellido" in persona ); // false
```

3.2. Asignación Desestructurante

En ocasiones, nos encontramos con que queremos **extraer** determinadas propiedades de un objeto o elementos de un *array* y asignarlas a variables independientes. Para ello se puede utilizar la **asignación desestructurante**.

En **arrays**, la asignación se realiza por **posición** y se utilizan **corchetes** `[ ]`. Es posible también utilizar el **operador rest** `...` para obtener el resto de parámetros en otro **array**.



```
let array1 = [ "valorA", "valorB", "valorC" ];

// En arrays las variables se asocian por posición
let [ a, b, c ] = array1;

console.log( a ); // valorA
console.log( b ); // valorB
console.log( c ); // valorC

// Ignorar un elemento
let [ d, , e ] = array1;
console.log( d ); // valorA
console.log( e ); // valorC

// Utilizar el operador rest: 'g' almacenará un array
let [ f, ...g ] = array1;
console.log( f ); // valorA
console.log( g ); // [ "valorB", "valorC" ]
```

En el caso de utilizar **objetos**, la asignación se realizará por **nombre** y mediante **llaves** `{ }`, y es posible también utilizar el operador **rest** `...`, que en este caso almacenará el resto de parámetros en un **objeto**.



Importante

JavaScript considera las llaves `{ }` como delimitadores de bloque. Por ello, si se utiliza una asignación desestructurante con variables previamente definidas, esto es, del modo `{ /* Variables */ } = OBJETO`, se generará un error, ya que se considera un bloque de código y no una asignación. Si se presenta el caso, se debe escribir la asignación entre paréntesis.

Si la asignación incluye también la declaración, del modo `let { /* Variables */ } = OBJETO`, no habrá problema.



```
let objeto1 = {
  pA: "valorA",
  pB: "valorB",
  pC: "valorC"
};

// En objetos las variables es asocian por nombre
let { pA, pB, pC } = objeto1;

console.log( pA ); // valorA
console.log( pB ); // valorB
console.log( pC ); // valorC

let resto;
// OJO: en este punto hemos definido 'resto' en la línea anterior
// No podemos poner 'let' en la línea siguiente porque 'pA' ya ha sido de
// Por tanto, debemos poner la expresión entre paréntesis
( { pA, ...resto } = objeto1 ); // Utilizar el operador rest: 'resto' al

console.log( pA );      // valorA
console.log( resto );   // {pB: "valorB", pC: "valorC"}
```

3.3. Referencia y Copia

Las variables **no almacenan objetos**, sino que **guardan referencias a ellos**. Así:

- Un objeto que no esté referenciado por ninguna variable **se elimina** de memoria.
- **No es posible copiar un objeto** realizando una **asignación** a otra variable: seguirá existiendo el mismo objeto referenciado por dos variables.

Para hacer una copia de un objeto se puede utilizar `Object.assign( OBJETO_DESTINO, OBJETO_ORIGEN )`.



```
let objeto1 = { nombre: "David" }; // Se crea un objeto. 'objeto1' apunta al objeto
let objeto2 = objeto1;             // NO SE CREA NINGÚN OBJETO NUEVO. 'objeto2' apunta al mismo objeto
let objeto3 = {};                  // Se crea un objeto vacío
let objeto4 = {};                  // Se crea otro objeto vacío

Object.assign( objeto3, objeto1 ); // Copia de 'objeto1' a 'objeto3'
Object.assign( objeto4, objeto1, { apellido: "Pérez" } ); // Copia de 'objeto1' a 'objeto4'

objeto1.nombre = "Juan"; // Cambio en 'objeto1'
objeto4.nombre = "Pablo"; // Cambio en 'objeto4'

console.log( objeto2.nombre ); // Juan ('objeto2' apunta al mismo objeto)
console.log( objeto3.nombre ); // David ('objeto3' apunta a un objeto diferente)

objeto1 = null; // 'objeto1' ya no apunta al objeto. El objeto sigue existiendo
objeto2 = null; // 'objeto2' ya no apunta al objeto. El primer objeto se elimina
```

3.4. Objetos Predefinidos y Métodos de Primitivas

Los tipos de datos **primitivos** pueden **convertirse a objetos**. Tal como veremos más adelante, JavaScript define un mecanismo de **herencia** basada en **prototipos**, de tal manera que los objetos pueden **acceder a propiedades definidas en sus prototipos**.

Esto tiene una serie de ventajas a la hora de tratar con determinados tipos de datos. Por ejemplo,

- al tratar con **strings** nos puede interesar calcular su longitud o convertir a minúsculas;
- al tratar con **números** nos puede interesar convertirlos a notación científica.

Para ello, JavaScript incorpora una serie de **objetos predefinidos en el lenguaje**, que actúan como prototipos que incluyen las **funcionalidades más comunes**. Entre ellos están los objetos **String** o **Number** (observa que tienen una mayúscula inicial, lo que los identifica como objetos y no como tipos primitivos), que se utilizan para tratar con sus tipos de datos asociados (**string** y **number**).

JavaScript incluye también el mecanismo para **convertir los tipos de datos primitivos a objetos complejos** que tengan acceso a esas funcionalidades. En el caso de los **strings**, la conversión se realiza de manera **automática**, mientras que en otros casos, como los **tipos de datos numéricos**, es necesario hacerlo de manera **explícita**. Una vez hecha la conversión para utilizar la funcionalidad correspondiente, el dato **se deja de nuevo como tipo de dato primitivo**.



Importante

Importante: Esta es la razón por la que podemos ejecutar un código como el siguiente:

```
let cadena = "Texto";  
console.log( cadena.length ); // 5
```

cadena almacena un tipo de datos primitivo, no un objeto; pero, al ejecutar **cadena.length**, JavaScript la convierte temporalmente a un **objeto** de tipo **String**,

que tiene acceso mediante herencia a la propiedad **length**, que devuelve la longitud de la cadena.

```
Number( 2.54 ).toExponential(); // '2.54e+0'
"texto".toUpperCase();          // "TEXT0"
Number( 25 ).toString();         // "25"

let cadena = "Esto es un texto";
cadena.indexOf( "x" );            // 13
cadena.includes( "texto" );       // true
cadena.startsWith( "Es" );        // true
cadena.endsWith( "Es" );          // false
```


3.5. Métodos de Arrays

Los **arrays** en JavaScript son **objetos**. Además, tienen acceso por herencia a una serie de funciones muy útiles.



```
let array1 = [ 'a', 'b', 'c' ];

array1.push( "d" );    // array1 = [ 'a', 'b', 'c', 'd' ]
array1.pop();          // array1 = [ 'a', 'b', 'c' ]
array1.shift();        // array1 = [ 'b', 'c' ] (la función devuelve 'a')
array1.unshift( "a" ); // array1 = [ 'a', 'b', 'c' ]

let array2 = [ 10, 11, 12, 13, 14 ];
array2.splice( 1, 2 ); // array2 = [ 10, 13, 14 ] (extrae 2 elementos)

let array3 = [ 1, 2, 3, 4, 5 ];
let array4 = array3.slice( 1, 4 ); // Extrae los elementos entre las posiciones 1 y 4
// array3 no se modifica
// array4 = [2, 3, 4]

let array5 = [ 1, 2 ];
let array6 = [ 8, 9 ];
let array7 = array5.concat( array6 ); // [1, 2, 8, 9]
// array5 y array6 no se modifican

array7.indexOf( 8 ); // 2
array7.includes( 1 ); // true
array7.includes( 3 ); // false
```

3.6. This

this en JavaScript es una palabra reservada que **se utiliza dentro de los métodos** de los objetos para hacer referencia al **objeto sobre el que se llama a dicho método en el lugar de ejecución, no en la definición**. El uso de **this** con frecuencia causa confusión, ya que su comportamiento es distinto del de otros lenguajes de programación.

```
let usuario = {
  nombre: "David",
  saluda: function() { // Lugar de definición de "saluda"
    return this.nombre;
  }
}

// Lugar de ejecución de "saluda"
// Se llama a la función sobre el objeto
usuario.saluda(); // David

// Lugar de ejecución de "saluda"
// Se llama a la función sin referencia al objeto
let intentaSaludar = usuario.saluda;
intentaSaludar(); // undefined
```

Como puede observarse, en apariencia, ambas ejecuciones de la función deberían producir el mismo resultado, dado que, en la definición, la función **saluda** hace referencia a **this**, que parece apuntar al objeto **usuario**. Sin embargo, vemos que solo funciona la primera versión, **usuario.saluda()**, mientras que la segunda, **intentaSaludar()**, pese a ser una variable que apunta a la misma función, no. La llamada **usuario.saluda()** realiza un enlace **implícito** (*implicit binding*).

Existen otras maneras de asociar la ejecución del método a un objeto, aparte del enlace implícito. Una posibilidad es utilizar el enlace **explícito** (*explicit binding*). Para ello, se utilizan las funciones **call**, **apply** o **bind**. Estas funciones están definidas en el **prototipo de cualquier función**, por lo que cualquier función puede hacer uso de ellas.



```
// Esta función tiene dos parámetros y además hace referencia a 'this'
function saludar( mensajeInicio, mensajeFinal ) {
    return mensajeInicio + this.nombre + mensajeFinal;
}

// Objeto sobre el que realizar pruebas
let usuario1 = { nombre: "David" };

// Ejecución de la función sin pasar referencia a ningún objeto.
saludar( "Hola ", ", ¿qué tal estás?" ); // Hola undefined, ¿qué tal estás?

// Ejecución de la función mediante 'call': los parámetros se pasan como
saludar.call( usuario1, "Hola ", ", ¿qué tal estás?" ); // Hola David, ¿qué tal estás?

// Ejecución de la función mediante 'apply': los parámetros se pasan en un array
saludar.apply( usuario1, [ "Hola ", ", ¿qué tal estás?" ] ); // Hola David, ¿qué tal estás?

// Enlace explícito de la función al objeto 'usuario1'
let saludaADavid = saludar.bind( usuario1 );

// La ejecución de la función se realiza sin pasar referencia. Sin embargo,
// ésta ya estaba pasada mediante 'bind'.
saludaADavid( "Hola ", ", ¿qué tal estás?" ); // Hola David, ¿qué tal estás?
```

Por último, es necesario volver a señalar que las **funciones flecha** no definen su propio **this**:

```
let usuario = {
    nombre: "David",
    saluda: () => { // Función 'saluda' definida como función flecha
        return this.nombre;
    }
}

// Lugar de ejecución de "saluda"
// Se llama a la función sobre el objeto
// 'this' no está definido, por ser una función flecha
usuario.saluda(); // undefined
```

¿Por qué esta diferencia? En ocasiones, el uso de **this** puede no funcionar como nosotros esperamos, sobre todo en el uso de **callbacks**. Consideremos el siguiente ejemplo con la función **setTimeout**, que es una función que pone en marcha un temporizador que ejecuta una función al terminar:

```
let usuario = {
  nombre: "David",
  saludaConRetardo: function() {
    setTimeout( function() {
      // Intenta mostrar el nombre del usuario
      console.log(this.nombre);
    }, 1000); // Retardo de 1000ms (1s)
  }
}

usuario.saludaConRetardo(); // undefined
```

Pese a llamar a la función haciendo referencia al objeto mediante **usuario.saludaConRetardo()**, la función no escribe en consola el nombre. ¿Por qué? Porque **setTimeout** es una función asíncrona que espera un tiempo y luego ejecuta la función pasada en su primer parámetro. Dicha llamada **se ejecuta sin referencia** al objeto, por lo que **this** no apunta al objeto que esperamos, sino que apunta al objeto global (**window**, en el navegador).

Para conseguir la funcionalidad deseada, podemos utilizar un **enlace explícito** (con **bind**, por ejemplo). Para ello debemos conseguir que la función **setTimeout** se ejecute sobre el objeto que queremos. Como la llamada a dicha función se realiza dentro del método **saludaConRetardo**, **this** es la referencia a dicho objeto. El código quedaría así:

```
let usuario = {
  nombre: "David",
  saludaConRetardo: function() {
    setTimeout( function() {
      console.log( this.nombre );
    }).bind( this ), 1000 );
    // Mediante 'bind(this)', conseguimos que la función
    // 'setTimeout' tenga como contexto el mismo objeto
    // que la función 'saludaConRetardo'
  }
}

// 'saludaConRetardo' tiene como contexto 'usuario' por enlace implícito en
```

```
// Como hemos utilizado 'bind', 'setTimeout' también tendrá el mismo contexto  
// Por tanto, 'this.nombre' apuntará a 'usuario.nombre', tal como queremos  
usuario.saludaConRetardo(); // David
```

Como puedes comprobar, el código anterior puede ser algo complejo de entender. Como alternativa, se puede utilizar una **función flecha**. El código quedaría así:

```
let usuario = {  
  nombre: "David",  
  saludaConRetardo: function() {  
    setTimeout( () => {  
      // Al utilizar una función flecha, 'setTimeout'  
      // no define su propio 'this'. JavaScript busca entonces  
      // 'this' en el ámbito exterior (ámbito léxico), es decir,  
      // el cuerpo de la función 'saludaConRetardo',  
      // que es justo lo que queremos  
      console.log(this.nombre);  
    }, 1000);  
  }  
}  
  
usuario.saludaConRetardo(); // David
```

Como puedes comprobar, el código con la función flecha es más fácil e intuitivo. La función flecha está definida en el lugar donde no queremos **this**, es decir, la función de **setTimeout**; la función **saludaConRetardo** sí que debe estar definida como función convencional.



Importante

Las reglas y el orden para determinar qué es **this** son las siguientes:

1. Si se utiliza el operador **new** (lo veremos más adelante), es el objeto devuelto por el constructor.
2. Si se utiliza enlace explícito (**call**, **apply** o **bind**), es el objeto especificado.
3. Si se utiliza enlace implícito, es el objeto especificado.

4. Si no se aplica nada de lo anterior, es el objeto global o **undefined** en modo estricto.
-

3.7. Prototipos y herencia

Como hemos comentado anteriormente, JavaScript es un **lenguaje orientado a objetos** que utiliza un sistema de **herencia basada en prototipos**.

Importante: En JavaScript **no hay clases**: hay **objetos enlazados a otros objetos, denominados prototipos**. Las propiedades de los prototipos **no se heredan**, es decir, no se copian a los objetos nuevos. Los objetos acceden a ellas a través de un mecanismo denominado **delegación**. Al tratar de acceder a una propiedad (incluyendo funciones) de un objeto, JavaScript busca primero si está definida en el objeto; si no lo está, busca en su prototipo.

Al crear un objeto, salvo casos especiales en que así lo establezcamos, se define una propiedad oculta que apunta a un objeto asociado: ese objeto es su **prototipo**. Cuando se intenta acceder a una **propiedad** de un objeto, JavaScript busca en primer lugar si está definida en dicho objeto; si no lo está, **busca en su prototipo**, y así sucesivamente en toda la cadena de dependencias, hasta llegar al objeto base (que suele ser `Object.prototype`). El siguiente código así lo demuestra:

```
let usuario = {
  nombre: "David"
}

// El método 'toLocaleString' funciona, pese a no haberlo definido
// Esta función está definida en 'Object.prototype'

usuario.toLocaleString(); // '[object Object]'
```

Este mecanismo nos permite acceder a una **jerarquía de objetos enlazados a otros objetos**. Ya hemos hablado de alguna de ellas, como la que nos permite trabajar con las funciones específicas de los **arrays**. Además de la jerarquía de objetos disponibles en el lenguaje, **podemos definir nuestra propia estructura de objetos enlazados mediante herencia prototípica**. Ahora bien, al tratarse de un tema complejo, no lo estudiaremos en profundidad.

Los mecanismos más utilizados para construir objetos basados en prototipos son dos: las **funciones constructoras** y los **objetos enlazados**. Existe también una tercera opción, la sintaxis `class`, que veremos en un epígrafe posterior.

A. FUNCIONES CONSTRUCTORAS

Una función constructora es una **función convencional** que define una serie de **propiedades y métodos**. Para hacer **referencia** a las propiedades y a otros métodos, hace uso del operador **this**. Y para crear objetos basados en ella utiliza el operador **new**.

```
// Función constructora. Suele escribirse la primera letra en mayúscula
function Usuario( nombre, apellidos ) {
    this.nombre = nombre;
    this.apellidos = apellidos;

    this.saludar = function() {
        console.log(`Hola, soy ${this.nombre} ${this.apellidos}`);
    }

    // 'nombre', 'apellidos' y 'saludar' son propiedades que se definirán
    // en los objetos que se creen a partir de la función constructora
}

// Se construye un objeto nuevo
let usuario1 = new Usuario( "David", "Pérez" );
usuario1.saludar(); // Hola, soy David Pérez

// Se construye un objeto nuevo
let usuario2 = new Usuario( "Jorge", "López" );
usuario2.saludar(); // Hola, soy Jorge López

// Se añade una nueva propiedad al prototipo
Usuario.prototype.despedir = function() {
    console.log( `${ this.nombre } ${ this.apellidos } se despide` );
}

// Los objetos tienen acceso a la nueva propiedad definida en el prototipo
usuario1.despedir(); // David Pérez se despide
usuario2.despedir(); // Jorge López se despide

// Se añade una nueva propiedad al prototipo
Usuario.prototype.direccion = "Desconocida";

console.log( usuario1.direccion ); // Desconocida
console.log( usuario2.direccion ); // Desconocida
```



```
// Se cambia una propiedad en el objeto
// La modificación se realiza en el objeto, no en el prototipo
usuario1.direccion = "Calle Nueva";

// JavaScript accede a la propiedad del objeto, que existe y tiene el valor
console.log( usuario1.direccion ); // Calle Nueva

// JavaScript accede a la propiedad del objeto, que no existe;
// A continuación accede al prototipo y la encuentra con valor "Desconocida"
console.log( usuario2.direccion ); // Desconocida
```

La función constructora define las propiedades que tendrán los objetos creados a partir de ella. Además, creará un enlace de dichos objetos a **NOMBRE_FUNCION.prototype**, que será el prototipo al que están enlazadas. Si se crea una **nueva propiedad** (incluyendo funciones) **en el prototipo**, los objetos **podrán acceder a ella**. Si se **modifica alguna propiedad definida en el prototipo**, **se copia al objeto** y no se modifica la del prototipo.

IMPORTANTE. El mecanismo de lectura y modificación de propiedades de los objetos es más complejo de lo que hemos visto. Si quieres más información, puedes buscar información sobre *getters* y *setters*.

B. OBJETOS ENLAZADOS

La segunda técnica para crear objetos basados en prototipos se basa en la función **Object.create** y utiliza exclusivamente objetos, no funciones constructoras. Por tanto, no utiliza tampoco el operador **new**.

```
// Objeto normal que será utilizado como prototipo
let Usuario = {
  nombre: "",
  apellido: ""
}

// Función para inicializar los datos
Usuario.setDatos = function( nombre, apellidos ) {
  this.nombre = nombre;
  this.apellidos = apellidos;
}
```

```
Usuario.saludar = function() {
    console.log( `Hola, soy ${ this.nombre } ${ this.apellidos }` );
}

// Se construyen objetos basados en el prototipo
let usuario1 = Object.create( Usuario );
let usuario2 = Object.create( Usuario );

// Se inicializan los objetos
usuario1.setDatos( "David", "Pérez" );
usuario2.setDatos( "Jorge", "López" );

// Función 'saludar'
usuario1.saludar(); // Hola, soy David Pérez
usuario2.saludar(); // Hola, soy Jorge López

// Se añade una nueva propiedad al prototipo
Usuario.despedir = function() {
    console.log( `${ this.nombre } ${ this.apellidos } se despide` );
}

// Los objetos tienen acceso a la nueva propiedad definida en el prototipo
usuario1.despedir(); // David Pérez se despide
usuario2.despedir(); // Jorge López se despide

// Se añade una nueva propiedad al prototipo
Usuario.direccion = "Desconocida";

console.log( usuario1.direccion ); // Desconocida
console.log( usuario2.direccion ); // Desconocida

// Se cambia una propiedad en el objeto
// La modificación se realiza en el objeto, no en el prototipo
usuario1.direccion = "Calle Nueva";

// JavaScript accede a la propiedad del objeto, que existe y tiene el valor
console.log( usuario1.direccion ); // Calle Nueva

// JavaScript accede a la propiedad del objeto, que no existe;
// A continuación accede al prototipo y la encuentra con valor "Desconocida"
console.log( usuario2.direccion ); // Desconocida
```

Este método es **más simple y esclarecedor** que el anterior, ya que no utiliza **new** y no hace referencia a **prototipe**. Como contrapartida, es menos común, sobre todo en código antiguo.

3.8. Prototipos nativos

JavaScript define un conjunto de prototipos **incluidos en el lenguaje**. Los más importantes son:

- **Object.prototype**. Es el prototipo base. Todos los objetos están enlazados con él (a no ser que se construya un objeto sin prototipo, que también es posible). Define algunos métodos básicos, como **toString()**, que convierte cualquier objeto a una cadena.
- **Function.prototype**. Es el prototipo con el que están enlazadas todas las funciones. Define, entre otros, los métodos **call**, **apply** y **bind**.
- Prototipos de primitivas. Ya hemos hablado de algunos de ellos, como **String** y **Number**.
- **Array.prototype**. Es el prototipo con el que están enlazados los **arrays**. Define varios métodos y propiedades muy útiles, como **length**, **pop**, **push**, **splice**, **concat**, etcétera.
- **Date.prototype**. Es el prototipo con el que se construyen los objetos de tipo **Date**. Define todos los métodos y propiedades relacionados con los objetos de fecha, como **getDate**, **getMinutes**, **getHours**, etcétera.



Importante

Puedes ver las propiedades definidas en los prototipos tanto en la consola de Node como en la del navegador. Para ello, escribe el nombre del prototipo (por ejemplo, **Date.prototype**) en la consola correspondiente. A continuación, verás la lista de métodos si estás en el navegador web; si estás en NodeJS podrás verlos pulsando la tecla **tabulador**.

3.9. Clases

JavaScript ha incorporado una **nueva sintaxis** para trabajar con objetos. Dicha sintaxis está creada para facilitar la tarea de las personas que vengan del mundo de Java u otro lenguaje orientado a objetos que utilice clases.

Importante: De nuevo, es importante recordar que en JavaScript no existen las clases en el sentido «clásico» (Java, C++). **Solo existen objetos relacionados mediante prototipos.**

A continuación, se muestra un ejemplo de definición de objetos mediante clases.

```
// Definición de clase

class Usuario {
  constructor(nombre, apellidos) {
    this.nombre = nombre;
    this.apellidos = apellidos;
  }

  // No hay coma entre las definiciones de los métodos
  saludar() {
    console.log(`Me llamo ${this.nombre} ${this.apellidos}`);
  }
}

// Creación de un objeto basado en la clase
// (En realidad, es un objeto cuyo prototipo es el objeto Usuario)
let usuario = new Usuario("David", "Pérez");
usuario.saludar(); // Me llamo David Pérez
```

También es posible crear objetos enlazados a otros mediante herencia prototípica. Para ello se utiliza **extends**, en línea con otros lenguajes que utilizan clases. De esta manera, da la impresión de que existe herencia clásica, aunque en realidad lo que existe es una **delegación**, tal como hemos comentado en apartados anteriores.

```
class Usuario {
  constructor( nombre, apellidos ) {
    this.nombre = nombre;
```

```

    this.apellidos = apellidos;
}

// 'saludar' está definido en 'Usuario.prototype'
saludar() {
    console.log( `Me llamo ${ this.nombre } ${ this.apellidos }` );
}
}

// Se crea un objeto, 'UsuarioEspecial.prototype', cuyo prototipo es 'Usuario'
class UsuarioEspecial extends Usuario {
    // 'cantar' está definido en 'UsuarioEspecial.prototype'
    cantar() {
        console.log( "Yo también sé cantar" );
    }
}

// Se crea un objeto cuyo prototipo es 'UsuarioEspecial'
let usuario = new UsuarioEspecial( "David", "Pérez" );

// JavaScript busca 'saludar' en el objeto ('usuario') y no lo encuentra
// A continuación busca en su prototipo, 'UsuarioEspecial' y tampoco lo encuentra
// Por último busca en el prototipo de 'UsuarioEspecial', que es 'Usuario' y
// 'saludar' está definido en 'Usuario.prototype'
// La ruta de búsqueda es: usuario -> UsuarioEspecial -> Usuario
usuario.saludar(); // Me llamo David Pérez

// JavaScript busca 'saludar' en el objeto ('usuario') y no lo encuentra
// A continuación busca en su prototipo, 'UsuarioEspecial', donde lo encuentra
// 'cantar' está definido en 'UsuarioEspecial.prototype'
// La ruta de búsqueda es: usuario -> UsuarioEspecial
usuario.cantar(); // También sé cantar

```

La sintaxis de clases también utiliza la palabra reservada **super** para **acceder a las propiedades del prototipo enlazado**. También se utiliza para **modificar los constructores** y **llamar al constructor del prototipo**.

```

class Usuario {
    constructor( nombre, apellidos ) {
        this.nombre = nombre;
        this.apellidos = apellidos;
    }
}

```

```
}

saludar() {
    console.log( `Me llamo ${ this.nombre} ${ this.apellidos }` );
}
}

// Se crea un objeto, 'UsuarioEspecial.prototype', cuyo prototipo es 'Usuario'
class UsuarioEspecial extends Usuario {
    // Se define un nuevo constructor con distintos parámetros
    constructor( nombre, apellidos, edad ) {
        // Se llama al constructor del prototipo
        // ES OBLIGATORIO hacerlo antes de poder utilizar 'this'
        // El constructor del prototipo define las propiedades 'nombre' y 'apellidos'
        super( nombre, apellidos )

        // Se crea la propiedad 'edad'
        this.edad = edad;
    }

    // Se sobrescribe el método 'saludar' para incluir funcionalidad adicional
    saludar() {
        // Se llama a la función 'saludar' del prototipo
        super.saludar();

        // Se añade la funcionalidad adicional
        console.log( `Y tengo ${ this.edad } años` );
    }
}

// Se crea un objeto cuyo prototipo es 'UsuarioEspecial' utilizando el nuevo constructor
let usuario = new UsuarioEspecial( "David", "Pérez", 35 );

// JavaScript busca 'saludar' en el objeto ('usuario') y no lo encuentra
// A continuación busca en su prototipo, 'UsuarioEspecial', donde lo encuentra
// 'saludar' llama a la función 'saludar' de 'Usuario' mediante 'super'
// 'saludar' ejecuta la segunda sentencia 'console.log'
usuario.saludar(); // Me llamo David Pérez
                  // Y tengo 35 años
```



Importante

Recuerda: si extiendes una clase y sobrescribes su constructor, debes llamar al constructor del prototipo mediante **super**.

3.10. JSON

JSON, *JavaScript Object Notation*, es un formato **basado en texto** para representar objetos. Dada su importancia para enviar y recibir datos entre clientes y servidores, JavaScript define los métodos:

- **JSON.stringify**. Para convertir objetos a JSON.
- **JSON.parse**. Para convertir JSON a objetos JavaScript.

```
let usuario = {  
  nombre: "David",  
  edad: 25,  
  permisos: [ "administrador", "editor" ]  
}  
  
// cadenaJson es un string  
let cadenaJson = JSON.stringify( usuario );  
  
// usuarioRecuperado es un objeto  
let usuarioRecuperado = JSON.parse( cadenaJson );
```


Test 3

3. Objetos y prototipos

1. Los objetos en JavaScript...

- Se crean siempre con `new`.
- Se crean a partir de una clase, igual que en Java.
- Se pueden crear de manera literal mediante `{ }`.
- Se deben crear siempre a partir de un constructor.

2. Si la variable `a` se asigna a un objeto y a continuación se ejecuta `let b = a; ...`

- `a` y `b` apuntan al mismo objeto.
- Se crea un objeto nuevo copia de `a` en `b`.
- Inicialmente, `a` y `b` apuntan al mismo objeto, pero si se modifica una propiedad de `b` no se modifica `a`.
- Inicialmente, `a` y `b` apuntan al mismo objeto, pero si se modifica una propiedad de `a` no se modifica `b`.

3. Indica cómo harías una copia de un objeto referenciado por la variable `a`.

- `let b = a;`
- `let b = Object.assign(a);`
- `let b = new a();`
- `let b = {}; Object.assign(b, a);`

4. ¿Qué mecanismos no define JavaScript para crear objetos basados en prototipos? (puede haber más de una respuesta válida).

- Clases.
- Funciones constructoras mediante `new`.
- Objetos enlazados mediante `Object.create`.
- Prototipos mediante `new Prototype`.

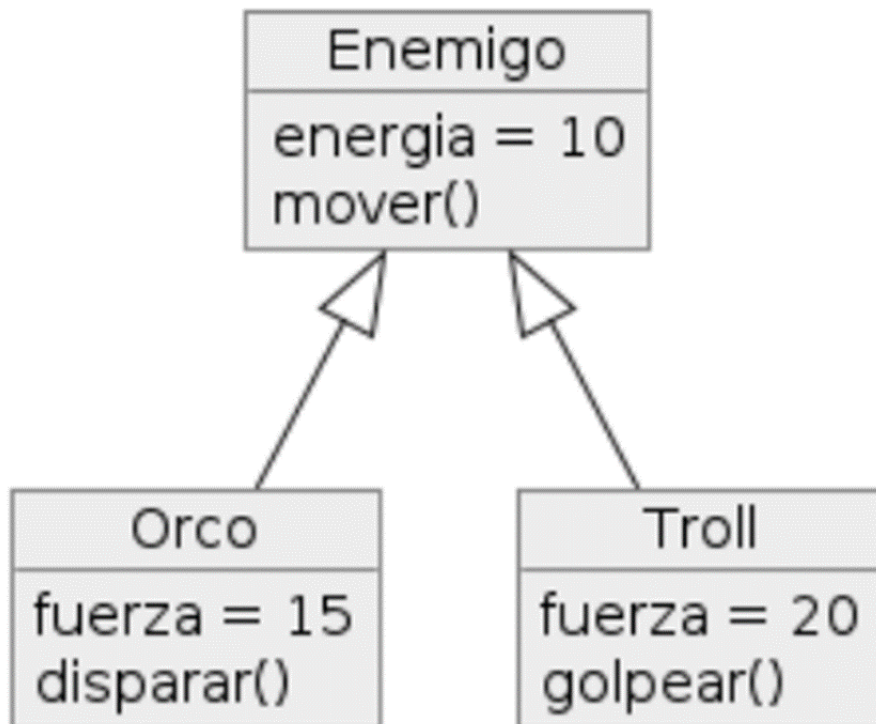
5. La sintaxis `class` de JavaScript permite definir clases con la misma filosofía que lenguajes como Java o C++.

- Verdadero
 - Falso
-

Actividad 2.5

Objetos y prototipos

Crea el código necesario para crear la siguiente estructura de prototipos:



Se deben poder crear objetos basados en cualquiera de los tres prototipos indicados. Puedes utilizar cualquiera de los tres sistemas estudiados:

- Sintaxis `class`.
 - Funciones constructoras.
 - Objetos enlazados con `Object.create`.
-

Actividad 2.6

This

Analiza el siguiente código e indica qué resultado se mostrará en las seis líneas indicadas, explicando el porqué de cada resultado.

```
function Persona(nombre, apellido) {
  this.nombre = nombre;
  this.apellido = apellido;
  this.saludar = function () {
    return `${this.nombre} ${this.apellido}`;
  }
}

let persona1 = new Persona("Ana", "Pérez");
let persona2 = new Persona("Lucía", "Martínez");
let persona3 = new Persona("Inés", "González");

let saludo1 = persona1.saludar.bind(persona3);
let saludo2 = persona2.saludar;

persona1.saludar();           // 1
persona2.saludar();           // 2
persona1.saludar.call(persona3); // 3
persona1.saludar.apply(persona2); // 4
saludo1();                     // 5
saludo2();                     // 6
```

Claves de resolución: `saludar` hace referencia a `this`. Recuerda que `this` es una referencia **dinámica** que no depende del lugar de declaración de la función, sino de **dónde y cómo se produzca la llamada a dicha función**.

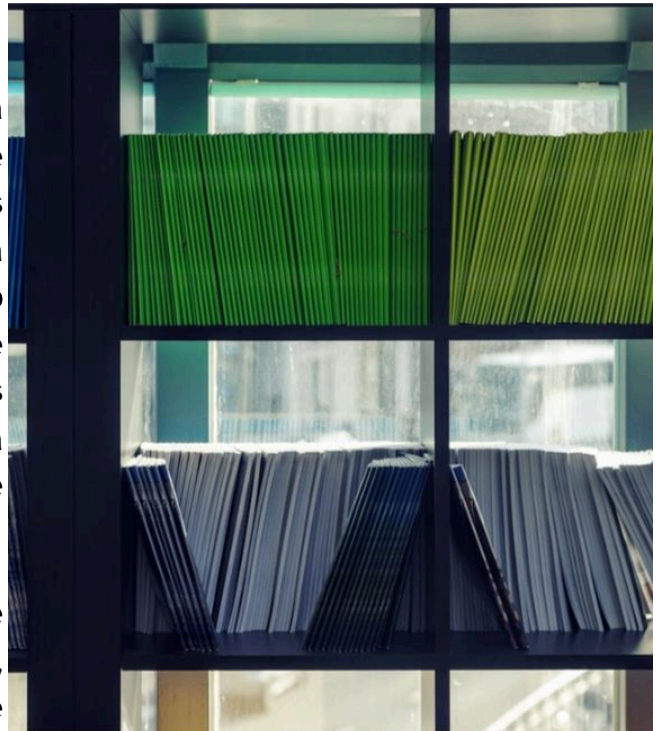
4. Módulos

Presentación

Cuando la complejidad de las aplicaciones aumenta, se hace necesario disponer de un mecanismo que nos permita dividir el código en diferentes archivos. Las ventajas de ello son variadas: mejor **organización**, **agrupación** de código relacionado con la misma funcionalidad, **disminución de conflictos de versiones** al poder trabajar diferentes personas en distintos archivos, etcétera.

Es muy probable también que sea necesario utilizar **librerías externas**. De nuevo, es deseable disponer de los medios para **cargar** dichas librerías de una forma **sencilla**, de tal manera que su **código interno** permanezca convenientemente **aislado** para evitar conflictos en nombres de variables o funciones, de forma que sea **accesible** únicamente la interfaz o API de dicha librería.

Este mecanismo recibe el nombre de sistema de **módulos**. Durante muchos años, JavaScript no ha tenido ningún sistema de módulos estándar en el navegador, sino que ha utilizado librerías externas creadas para ello o ha adoptado el sistema creado para NodeJS. Finalmente, el estándar EcmaScript ha desarrollado su propio sistema, que ha sido incluido en JavaScript y en la actualidad es compatible con la mayoría de los navegadores, así como NodeJS.



4.1. Tipos de módulos

Los programas de JavaScript tradicionalmente se han cargado a través de ficheros externos mediante etiquetas `<script>` en el navegador. En los inicios, dichos ficheros definían un conjunto de funciones y variables que se incorporaban **directamente al objeto global**. Esta situación provocaba los siguientes inconvenientes:

- Se podían producir **conflictos** entre nombres de funciones o variables definidas con el mismo nombre en distintos archivos. Esto se solucionaba incorporando un **prefijo** a esos nombres para evitarlo.
- Se podían **modificar** las variables y funciones definidas en otros ficheros, intencionadamente o no, con el consiguiente problema de seguridad que acarreaba.

Para evitar estos problemas, se desarrolló el concepto de **módulo**. Un módulo es un **fichero JavaScript** que permite definir **qué objetos (en forma de variables o funciones) se exportarán** para que puedan ser utilizados por el objeto global u otros módulos. El resto permanecerá oculto y no será modificable desde el exterior.

Se han desarrollado distintos **sistemas de módulos**. A continuación, se indican algunos de ellos:

- **CommonJS**. Es el que se definió para NodeJS.
- **AMD**. Utilizado en navegadores por librerías de carga de módulos.
- **UMD**. Sistema universal compatible con **CommonJS** y **AMD**.
- **ES6**. Sistema de módulos **incorporado al estándar EcmaScript**. Es el sistema llamado a reemplazar a todos los demás. Es compatible con los navegadores actuales. En NodeJS se puede utilizar si los archivos utilizan la extensión **.mjs** en lugar de **.js**.

En este apartado estudiaremos los módulos **ES6**, a los que nos referiremos simplemente como **módulos**.

Importante: Dado que los módulos ES6 son de reciente incorporación al lenguaje, todavía es posible encontrar gran cantidad de librerías que utilizan alguno de los otros sistemas no estándar.

4.2. "import" y "export"

El sistema de módulos define dos directivas en el lenguaje: **import** y **export**:

- **import** se utiliza para importar funcionalidad desde otros módulos. Solo se podrán importar objetos de un módulo si han sido exportadas por este.
- **export** se utiliza para definir qué objetos de los definidos en el módulo serán **accesibles desde el exterior**, es decir, podrán ser importados por el código que utilice dicho módulo.

A continuación, se muestra el código de un módulo, guardado en el archivo **miLibreria.js**:

```
// Fichero miLibreria.js

function saludar(usuario) {
    console.log(`Hola, ${usuario}`);
}

function secreto() {
    console.log("Sssssh... es un secreto...");
}

function despedir(usuario) {
    secreto();
    console.log(`Adiós, ${usuario}`);
}

// Exportación de objetos.
// Se exporta un objeto con dos propiedades.
// Cada propiedad hace referencia a una de las funciones definidas arriba.
// La función 'secreto' no se exporta, por lo que no es visible fuera del módulo.
// sin embargo, dentro del módulo sí que puede utilizarse.
export {
    saludar,
    despedir
}
```

Seguidamente, se muestra el código de un programa, almacenado en el fichero **principal.js**, que utiliza el módulo definido anteriormente:

```
// Fichero principal.js

// Importación de las funciones exportadas por el módulo
// Mediante esta sintaxis se pueden indicar las funciones que se deseen importar
// que no tienen por qué ser todas
import { saludar, despedir } from './miLibreria.js';

// A partir de aquí se pueden utilizar las funciones importadas, 'saludar' y
saludar("Juan"); // Hola, Juan
despedir();       // "Sssssh... es un secreto..."
                  // Adiós, Juan
```

Si se desea importar toda la funcionalidad de un módulo, es posible utilizar la siguiente sintaxis:

```
// Fichero principal.js
import * as miLibreria from './miLibreria.js';

// El objeto 'miLibreria' tiene definidas todas las funciones exportadas por
miLibreria.saludar("Juan"); // Hola, Juan
miLibreria.despedir();       // "Sssssh... es un secreto..."
                             // Adiós, Juan
```

Importante: Existen otras posibilidades para exportar objetos utilizando módulos, como la opción **default**.

4.3. Módulos en HTML

Para trabajar con **módulos en HTML** y utilizar **import** para cargarlos en nuestros programas es necesario utilizar el atributo **type="module"** en la etiqueta **<script>**.

```
<script type="module">
  import * as miLibreria from './miLibreria.js';

  miLibreria.saludar("Juan");
  miLibreria.despedir();
</script>
```

Importante: **import** siempre debe definir una **ruta** (absoluta o relativa) **al archivo del módulo** que se pretenda importar: si el archivo **miLibreria.js** se encuentra en la misma carpeta que la página HTML se debe indicar **./** para señalar la ruta al directorio actual; el código **import * as miLibreria from 'miLibreria.js'** no funcionará.

Importante: Los **módulos** solo se cargan **a través de HTTP**, no a través del sistema de ficheros. Por tanto, para hacer pruebas no es posible lanzar la página haciendo doble clic sobre el fichero HTML, sino que se debe cargar a través de un servidor web (por ejemplo, usando la extensión **Live Server** de Visual Studio Code). Recuerda que, si la barra de direcciones comienza por **file://** al cargar una página web, estás realizando la carga a través del sistema de ficheros.

4.4. Librerías externas: CDN

Mediante el sistema de módulos, podemos acceder a un gran número de **librerías externas** creadas por otras personas. JavaScript tiene una de las comunidades de desarrolladores más grandes del mundo, por lo que la oferta es muy extensa.

La mayoría de las librerías más utilizadas ofrecen compatibilidad con **import**, aunque no todas: es posible que algunas de ellas nos pidan que las utilicemos cargándolas en el objeto global mediante una etiqueta **<script>** convencional.

En cuanto a la modalidad de carga, nos encontraremos con dos posibilidades:

- **Descarga** del fichero (o ficheros) para almacenarlos junto con nuestro código.
- Acceso al fichero (o ficheros) a través de una **URL** sin necesidad de descargar. En este caso, la URL suele apuntar a un **CDN**.

Un **CDN**, o *content delivery network*, es un **servicio distribuido de alojamiento de ficheros**. Su principal función es **garantizar la descarga de dichos ficheros de manera óptima**. Para ello, utiliza una arquitectura distribuida de servidores: la descarga, que se produce de manera transparente al usuario, se realiza desde el servidor que ofrezca mejor servicio, bien por distancia geográfica, por latencia o por carga actual del sistema. Encontrarás algunos de los CDN más famosos en los enlaces web al final de la unidad.

La elección de utilizar un CDN frente a descargar la librería para almacenarla en nuestro servidor tiene una serie de **ventajas**:

- Ahorro de espacio.
- Ahorro de ancho de banda en nuestro servidor.
- Mejor comportamiento en la descarga.
- Simplicidad a la hora de realizar actualizaciones de versiones.

También tiene desventajas, como la **dependencia de un servidor externo**: si el CDN se cae, o deja de dar servicio, perderemos el acceso a las librerías.

Test 4

4. Módulos

1. Indica qué sistema de módulos es el que fue creado para NodeJS.

- AMD.
- CommonJS.
- UMD.
- ES6.

2. Indica la manera correcta de importar todas las propiedades que exporta un módulo.

- `import * as miLibreria from 'miLibreria.js'`
- `import * from './miLibreria.js'`
- `import * from 'miLibreria.js'`
- `import * as miLibreria from './miLibreria.js'`

3. Para importar un script de tipo módulo en HTML hay que utilizar... (las respuestas van entre < y >)

- `script`
- `script type="javascript"`
- `script type="module"`
- `script type="javascript-module"`

4. Utilizar un CDN ahorra ancho de banda en el servidor donde está alojada la aplicación web.

- Verdadero
- Falso

5. Utilizar un CDN hace que sea necesario descargar las librerías que utilice nuestra aplicación cada vez que queremos utilizar una versión actualizada.

- Verdadero
 - Falso
-

Actividad 2.7

Módulos

Crea un módulo denominado en un archivo con extensión `.js` y almacena el código JavaScript creado en la Actividad 2.5 . Deja únicamente las definiciones de los prototipos y expórtalas para que puedan ser utilizadas mediante **`import`**.

A continuación, crea una página HTML que contenga un *script* que importe dicho módulo y cree un enemigo de cada tipo utilizando las funciones importadas.

Actividad 2.8

Importación de librerías externas

Se proporciona un código HTML de partida que utiliza una librería externa para mostrar una gráfica. Dicha librería es `chart.js`. Todo el código está preparado para funcionar, salvo por un detalle: **falta importar el código** de dicha librería para que pueda funcionar.

Modifica el código del archivo para **importar** la librería mediante el **sistema de módulos ES6** y que pueda funcionar la página. Está alojado en el CDN `jsdelivr`; concretamente, en la página <https://www.jsdelivr.com/package/npm/chart.js?path=dist> **<https://www.jsdelivr.com/package/npm/chart.js?path=dist>**.

A continuación, se indica la lista de objetos que debes importar: `Chart`, `LinearScale`, `BarController`, `CategoryScale`, `BarElement`.

Claves de resolución.

Al acceder a la URL proporcionada, verás varios enlaces a diferentes archivos. Dado que el sistema de módulos todavía no está implantado al 100 %, la mayoría de las librerías ofrecen enlaces para realizar la carga mediante el sistema tradicional de importación al objeto global. En el caso de que proporcionen archivos compatibles con el sistema de módulos, estos tienen la extensión `.esm.js`. Por tanto, deberemos copiar la ruta al archivo `chart.esm.js`.

El código de partida es el siguiente:

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Uso de librerías externas</title>
</head>

<body>
  <div>
    <canvas id="myChart" width="400" height="400"></canvas>
  </div>
```

```
<script type="module">
  // TODO: Importar funciones de la librería externa a través del C

  Chart.register(LinearScale, BarController, CategoryScale, BarElement);
  const ctx = document.getElementById('myChart');
  const myChart = new Chart(ctx, {
    type: 'bar',
    data: {
      labels: ['Red', 'Blue', 'Yellow', 'Green', 'Purple', 'Orange'],
      datasets: [{
        label: '# of Votes',
        data: [12, 19, 3, 5, 2, 3],
        backgroundColor: [
          'rgba(255, 99, 132, 0.2)',
          'rgba(54, 162, 235, 0.2)',
          'rgba(255, 206, 86, 0.2)',
          'rgba(75, 192, 192, 0.2)',
          'rgba(153, 102, 255, 0.2)',
          'rgba(255, 159, 64, 0.2)'
        ],
        borderColor: [
          'rgba(255, 99, 132, 1)',
          'rgba(54, 162, 235, 1)',
          'rgba(255, 206, 86, 1)',
          'rgba(75, 192, 192, 1)',
          'rgba(153, 102, 255, 1)',
          'rgba(255, 159, 64, 1)'
        ],
        borderWidth: 1
      }]
    },
    options: {
      scales: {
        y: {
          beginAtZero: true
        }
      }
    }
  });
</script>
</body>

</html>
```

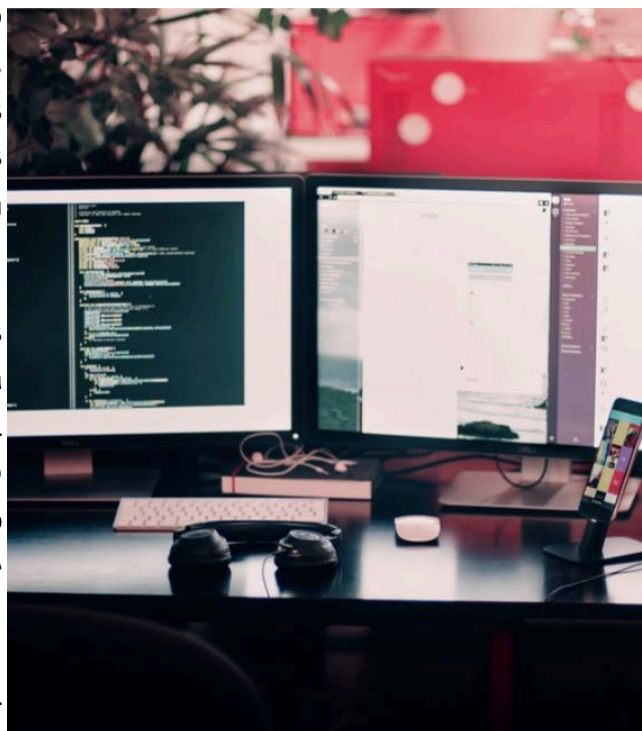
5. Programación funcional

Presentación

JavaScript es compatible con el paradigma de **programación funcional**. Este estilo de programación tiene como característica principal el uso de **funciones** que devuelven datos que, a su vez, pueden ser utilizados por otras funciones. Las funciones suelen estar altamente especializadas y, en la medida de lo posible, minimizan los efectos secundarios, es decir, **no provocan cambios** en variables externas ni en los parámetros que se les pasan, sino que se limitan a **devolver un resultado nuevo**.

En la programación funcional, las funciones se consideran **ciudadanas de primera clase** (*first-class citizens*): reciben el mismo tratamiento que cualquier otro tipo de datos, por lo que se pueden **pasar como parámetros** a otras funciones y se pueden **almacenar en variables**.

En la programación funcional es habitual utilizar funciones para realizar tareas que en la programación procedimental se realizarían con **bucles**. Los usos más habituales están relacionados con el tratamiento de los datos almacenados en **arrays**, como la búsqueda, filtrado, transformación o cálculo de datos agregados.



5.1. Fundamentos: la función Filter

La programación funcional es un paradigma de programación en el que las funciones son las únicas estructuras para crear programas. Así, las funciones pueden asignarse a variables y pasarse como parámetros de otras funciones. JavaScript soporta este paradigma de programación, de igual forma que soporta el paradigma de programación orientada a objetos. A lo largo de esta unidad hemos visto algunos ejemplos de programación funcional, como los **callbacks**. En este apartado profundizaremos algo más, concretamente en el apartado de **procesamiento de arrays**. Es muy habitual que los programas trabajen con arrays para realizar modificaciones sobre ellos, como, por ejemplo:

- Realizar un **filtrado** para obtener un subconjunto de elementos que cumplan una condición.
- **Buscar** un determinado elemento.
- Modificar un *array* para **transformarlo** en otro con otras propiedades derivadas de las primeras.
- Realizar un **cálculo** con los elementos del array.
- Realizar **agrupaciones** de elementos del array en función de alguna condición.

Estas operaciones suelen realizarse de manera **iterativa**, utilizando **bucles que recorran todos los elementos del array**. Por ejemplo, para realizar un **filtrado** utilizaríamos un código parecido a este:

```
let datos = [14, 22, 16, 3, 35];

// Obtener los elementos menores que 15
let resultado = [];

for (let dato of datos) {
  if (dato < 15) {
    resultado.push(dato);
  }
}

console.log(resultado); // [14, 3]
```

El código anterior es sencillo, y estamos familiarizados con él. Sin embargo, la estructura utiliza un **for** en el que se modifica la variable **resultado** de manera indirecta: no hay una clara asignación a **resultado**, algo del tipo **resultado=VALOR_RESULTADO_DEL_FILTRADO**. A continuación, se muestra su **alternativa** funcional:

```
let datos = [14, 22, 16, 3, 35];

// Obtener los elementos menores que 15
let resultado = datos.filter(
  function (dato) {
    return dato < 15;
  });

console.log(resultado); // [14, 3]
```

En este segundo caso sí que tenemos una asignación sobre la variable **resultado**, lo que facilita su comprensión. Quizá el hecho de tener una función anónima complica un poco la visualización si no estamos acostumbrados. Para aclararlo aún más, mostramos una tercera versión:

```
let datos = [14, 22, 16, 3, 35];

// Función que comprueba si un dato es menor que 15
// Se ha utilizado una función flecha en este caso
// Como solo tiene un parámetro, se pueden omitir los paréntesis
// Como solo tiene una línea en el cuerpo, se pueden omitir las llaves y el
// Devuelve true si se cumple la condición y false en caso contrario
let menorQue15 = dato => dato < 15;

// Obtener los elementos menores que 15
let resultado = datos.filter(menorQue15);

console.log(resultado); // [14, 3]
```

En este último código se ve claramente que **resultado** es el **resultado** de realizar una operación **filter** con una condición **menorQue15**. La función **menorQue15** es una función que devuelve únicamente **true** o **false** y que **se pasa como parámetro** a otra función. A este tipo de funciones se las denomina **funciones predicado**.

Importante: Las **funciones flecha** son muy adecuadas para crear funciones predicado, ya que normalmente estas funciones no suelen hacer uso de **this**. Por supuesto, también pueden utilizarse las funciones convencionales.

Importante: La función que se pasa como parámetro a **filter** debe aceptar **tres parámetros**: el **elemento** que se esté procesando (único parámetro **obligatorio**), la **posición**

y el ***array* original**. Sin embargo, es muy habitual que se declare únicamente el primero, que corresponde al elemento del *array* que se está procesando.

A continuación, veremos otras funciones que incorpora JavaScript además de ***filter*** para trabajar con *arrays* de manera funcional.

5.2. Find

Devuelve el **primer elemento** que cumpla la condición pasada en la función. A continuación, se muestra un ejemplo con un *array* de objetos:

```
let usuarios = [
  { nombre: "David", apellido: "Pérez", nif: 1 },
  { nombre: "Juan", apellido: "Martínez", nif: 2 },
  { nombre: "José", apellido: "Fernández", nif: 3 }]

let usuarioConNif2 = usuarios.find(item => item.nif == 2);
console.log(usuarioConNif2);    // {nombre: 'Juan', apellido: 'Martínez', nif: 2}
```

Como podemos ver, la función que se pasa como parámetro a **find** es de nuevo una **función predicado**, ya que devuelve únicamente **true** o **false**. Como en el caso de **filter**, dicha función puede aceptar tres parámetros (elemento, índice y *array*), aunque se suele utilizar solo con el primero.

5.3. ForEach

Permite **ejecutar una función en cada uno de los elementos del *array***. Por ejemplo, si queremos ejecutar la función **alert** en cada elemento de un array, podemos ejecutar:

```
let datos = ["a", "b", "c", "d"];
datos.forEach(alert);
```

Como en el caso anterior, la función de *callback* acepta tres parámetros (elemento, posición y *array* original). Un ejemplo completo sería:

```
let datos = ["a", "b", "c", "d"];
datos.forEach(function (item, indice, array) {
    console.log(`Procesando el elemento ${item}, en la posición ${indice}. El array es a,b,c,d.`);
});

// Resultado:
// Procesando el elemento a, en la posición 0. El array es a,b,c,d.
// Procesando el elemento b, en la posición 1. El array es a,b,c,d.
// Procesando el elemento c, en la posición 2. El array es a,b,c,d.
// Procesando el elemento d, en la posición 3. El array es a,b,c,d.
```

Importante: En este caso, la función que se pasa como parámetro no es una función predicado, ya que no devuelve **true/false**. Así, hablaríamos de una función de *callback* únicamente.

5.4. Map

Se utiliza para realizar **transformaciones de arrays**. A continuación, se muestra un ejemplo:

```
let numeros = [1, 3, 5, 7];
let dobleNumeros = numeros.map(
  function (numero) {
    return numero * 2;
  });

// No se modifica el array original
console.log(numeros);           // [1, 3, 5, 7]
console.log(dobleNumeros);      // [2, 6, 10, 14]
```

Hay que tener cuidado al trabajar con **arrays de objetos**, ya que la función que se pasa a **map** puede realizar **modificaciones sobre dichos objetos**. Si esto ocurre, es posible que se devuelva un nuevo array, cuyos elementos **apunten a los mismos objetos originales**.

```
let usuarios = [
  { nombre: "David", apellido: "Pérez", nif: 1 },
  { nombre: "Juan", apellido: "Martínez", nif: 2 },
  { nombre: "José", apellido: "Fernández", nif: 3 }
]

let usuariosMod = usuarios.map(
  usuario => {
    // Se modifica el objeto original
    usuario.nombreCompleto = `${usuario.nombre} ${usuario.apellido}`;
    // Se devuelve una referencia al mismo objeto original
    return usuario;
  })

console.log(usuariosMod[0].nombreCompleto); // David Pérez
console.log(usuariosMod[1].nombreCompleto); // Juan Martínez
console.log(usuariosMod[2].nombreCompleto); // José Fernández

// Comprobamos que el array original apunta a los mismos objetos
console.log(usuarios[0].nombreCompleto); // David Pérez
console.log(usuarios[1].nombreCompleto); // Juan Martínez
console.log(usuarios[2].nombreCompleto); // José Fernández
```

```
// Comprobamos que se apunta al mismo objeto:
// Cambiamos el valor a través de 'usuariosMod'
usuariosMod[0].nombreCompleto = "aa";

// Comprobamos el valor a través de 'usuarios'
console.log(usuarios[0].nombreCompleto); // aa
```

El código anterior no es necesariamente malo: simplemente hay que tener en cuenta su funcionamiento. Si no nos importa modificar el *array* original (o es lo que pretendemos), entonces, perfecto; si lo que queremos es que **map** devuelva un *array* de objetos distintos, deberíamos cambiar el programa.

```
let usuarios = [
  { nombre: "David", apellido: "Pérez", nif: 1 },
  { nombre: "Juan", apellido: "Martínez", nif: 2 },
  { nombre: "José", apellido: "Fernández", nif: 3 }
]

let usuariosMod = usuarios.map(
  usuario => {
    // Se crea un nuevo objeto vacío
    let nuevoUsuario = {};
    // Se copia el objeto original al nuevo objeto
    Object.assign(nuevoUsuario, usuario);
    // Se modifica el nuevo objeto
    nuevoUsuario.nombreCompleto = `${nuevoUsuario.nombre} ${nuevoUsuario.apellido}`;
    // Se devuelve el objeto nuevo
    return nuevoUsuario;
  })

console.log(usuariosMod[0].nombreCompleto); // David Pérez
console.log(usuariosMod[1].nombreCompleto); // Juan Martínez
console.log(usuariosMod[2].nombreCompleto); // José Fernández

// Comprobamos que el array original no se ha modificado
console.log(usuarios[0].nombreCompleto); // undefined
console.log(usuarios[1].nombreCompleto); // undefined
console.log(usuarios[2].nombreCompleto); // undefined

// Comprobamos que se apunta a objetos distintos:
// Cambiamos el valor a través de 'usuariosMod'
usuariosMod[0].nombreCompleto = "aa";
usuariosMod[0].nombre = "Distinto";
```

```
// Comprobamos el valor a través de 'usuarios'  
console.log(usuarios[0].nombreCompleto); // undefined  
console.log(usuarios[0].nombre); // David
```

Como en el caso anterior, la función de *callback* acepta tres parámetros, aunque es muy habitual que se declare únicamente el primero, que corresponde al elemento del *array* que se está procesando.

5.5. Reduce

La última función que veremos es la más compleja. Esta función se encarga de recorrer todos los elementos del *array* para **generar un único valor**. Se suele utilizar para realizar **cálculos** (totales, medias, etcétera) o **agrupaciones** de datos.

Importante: `reduce` devuelve un único valor, pero puede ser un objeto o un *array*. no tiene por qué ser un tipo de datos sencillo.

`reduce` admite dos parámetros:

1. Una **función de *callback***, que admite **cuatro parámetros**: el **resultado de la ejecución anterior** del *callback* (habitualmente llamado **acumulador**) y los tres que conocemos de las funciones que hemos estudiado antes: el valor actual del *array*, el índice del elemento que se está procesando y el *array*. Únicamente son **obligatorios los dos primeros**.
2. Un **valor inicial**, que se utilizará en la **primera ejecución del *callback*** (ya que la primera vez que se ejecuta el *callback* no hay un resultado de la ejecución anterior). Este parámetro es opcional, aunque es muy habitual que se proporcione con un valor vacío: `"", 0, []` o `{}`.

Veamos un ejemplo de funcionamiento para calcular la suma total la media de los elementos de un *array*.

```
let numeros = [2, 5, 4];
let total = numeros.reduce(
  // Primer parámetro: función de callback
  function (acc, numero) {
    return acc + numero;
  },
  // Segundo parámetro: valor inicial
  0);

console.log(total); // 11
```

Los pasos de ejecución serían los siguientes:

1. **acc = 0** (valor inicial). La función devuelve $0 + 2 = 2$.
2. **acc = 2** (resultado de la ejecución anterior). La función devuelve $2 + 5 = 7$.
3. **acc = 7** (resultado de la ejecución anterior). La función devuelve $7 + 4 = 11$.

Por último, veamos un ejemplo más complejo para realizar una **agrupación**, que consiste en clasificar un listado de coches por color. El resultado se devolverá en forma de **objeto**, y sus propiedades serán los colores de los coches existentes.

```
// Listado de coches
let coches = [
  { matricula: "ABC1", color: "rojo" },
  { matricula: "ABC2", color: "verde" },
  { matricula: "ABC3", color: "azul" },
  { matricula: "ABC4", color: "rojo" }
]

let clasificacion = coches.reduce(
  function (acc, coche) {
    // Hay que comprobar si existe una entrada en el objeto para el color
    // El operador '||' ('or' lógico) se comporta como un condicional:
    // si el primer operando está definido, se devuelve; si no, se devuelve el segundo
    // En este caso, si existe 'acc[coche.color]', se utiliza;
    // si no, se crea un objeto nuevo con las propiedades 'cuenta' y 'listado'
    acc[coche.color] = acc[coche.color] || { cuenta: 0, listado: [] };
    // Se incrementa la cuenta de coches de ese color
    acc[coche.color].cuenta++;
    // Se añade el coche al listado del color correspondiente
    acc[coche.color].listado.push(coche);
    return acc;
  }, {}); // El valor inicial es un objeto vacío

console.log(clasificacion);
// {
//   rojo: { cuenta: 2, listado: [ [Object], [Object] ] },
//   verde: { cuenta: 1, listado: [ [Object] ] },
//   azul: { cuenta: 1, listado: [ [Object] ] }
// }

// Cuántos coches hay de color rojo
console.log(clasificacion.rojo.cuenta); // 2
// Cuántos coches hay de color verde
console.log(clasificacion.verde.cuenta); // 1
// Cuántos coches hay de color azul
console.log(clasificacion.azul.cuenta); // 1

// Listado de coches de color rojo
console.log(clasificacion.rojo.listado);
// [
//   { matricula: 'ABC1', color: 'rojo' },
```



```
// { matricula: 'ABC4', color: 'rojo' }  
// ]
```

Los pasos de ejecución serían los siguientes:

1. **acc**={ } (valor inicial). La función comprueba si existe **acc["rojo"]**. Como no existe, lo crea mediante **acc["rojo"]={cuenta:0,listado:[]}**. A continuación, incrementa **acc["rojo"].cuenta** y añade el primer objeto coche a **acc["rojo"].listado**. Por último, devuelve **acc**.
2. **acc** es un objeto con una propiedad definida, **"rojo"**. La función comprueba si existe **acc["verde"]**. Como no existe, lo crea mediante **acc["verde"]={cuenta:0,listado:[]}**. A continuación, incrementa **acc["verde"].cuenta** y añade el segundo objeto coche a **acc["verde"].listado**. Por último, devuelve **acc**.
3. **acc** es un objeto con **dos** propiedades definidas, **"rojo"** y **"verde"**. La función comprueba si existe **acc["azul"]**. Como no existe, lo crea mediante **acc["azul"]={cuenta:0,listado:[]}**. A continuación, incrementa **acc["azul"].cuenta** y añade el tercer objeto coche a **acc["azul"].listado**. Por último, devuelve **acc**.
4. **acc** es un objeto con tres propiedades definidas, **"rojo"**, **"verde"** y **"azul"**. La función comprueba si existe **acc["rojo"]**. Como **sí existe, utiliza el objeto existente** mediante **acc["rojo"]=acc["rojo"]**. A continuación, incrementa **acc["rojo"].cuenta** (que en este caso vale **1**, no **0**, porque ya se había incrementado en el paso 1, y añade el cuarto objeto coche a **acc["rojo"].listado** (que ya tenía incluido el primer coche). Por último, devuelve **acc**.

Importante: La función **reduce** es la más potente de todas las funciones que hemos estudiado en este apartado. Tanto es así que se pueden implementar todas las demás a partir de ella.

Test 5

5. Programación funcional

1. Señala las características que definen una función predicado (puede haber más de una respuesta válida).

- Devuelve únicamente true o false.
- Se construye exclusivamente como función flecha.
- Se suele utilizar como parámetro de otra función.
- Se denomina también función de callback.

2. ¿Qué hay que pasar a la función `filter` como parámetro?

- Un objeto.
- Un array.
- Una función.
- Un string.

3. ¿Qué funciones reciben como parámetro funciones predicado?

- `filter` y `map`.
- `filter` y `find`.
- `map` y `reduce`.
- `find` y `forEach`.

4. De entre las funciones que se indican a continuación, ¿cuál de ellas puede utilizarse para implementar todas las demás?

- `map`.
- `find`.
- `filter`.
- `reduce`.

5. Si quiero calcular el número de objetos de un array que tienen una determinada propiedad, ¿qué función podría devolverme dicha información directamente?

- `map`.
- `reduce`.
- `find`.
- `filter`.

Actividad 2.9

Trabajo con arrays de objetos

Dado el siguiente *array* de objetos, realiza las siguientes operaciones mediante programación funcional:

- Obtén los coches de la marca «Seat».
- Obtén los coches cuyo precio sea menor que 25000.
- Encuentra el primer coche cuyo precio sea mayor que 20000.
- Transforma el *array* en otro con los mismos campos más uno adicional, denominado **premium**. Si el precio es mayor de 20000, **premium=true**; en caso contrario, **premium=false**.
- Calcula el precio medio de los coches de la marca «Seat».

```
let coches = [  
  { id: 1, marca: "Seat", modelo: "Ibiza", precio: 10000 },  
  { id: 2, marca: "Seat", modelo: "Ateca", precio: 18000 },  
  { id: 3, marca: "Volkswagen", modelo: "Golf", precio: 21000 },  
  { id: 4, marca: "Kia", modelo: "Niro", precio: 30000 },  
];
```

Actividad 2.10

Cálculo de medias (variante funcional)

Vamos a resolver la Actividad 2.3 del apartado de **Funciones** utilizando **código funcional en lugar de bucles**. El objetivo es el mismo: crear una función que calcule la **media aritmética** de un conjunto variable de valores pasados como parámetros.

Claves de Resolución: La función tiene que procesar un conjunto variable de parámetros, por lo que se deberán utilizar de nuevo los parámetros **rest**. En cuanto a la elección de la función que se deberá utilizar, hay que tener en cuenta que queremos operar sobre un listado (*array*) y generar como resultado un **único valor**. ¿Cuál de las funciones estudiadas en este apartado está especializada en esa tarea?

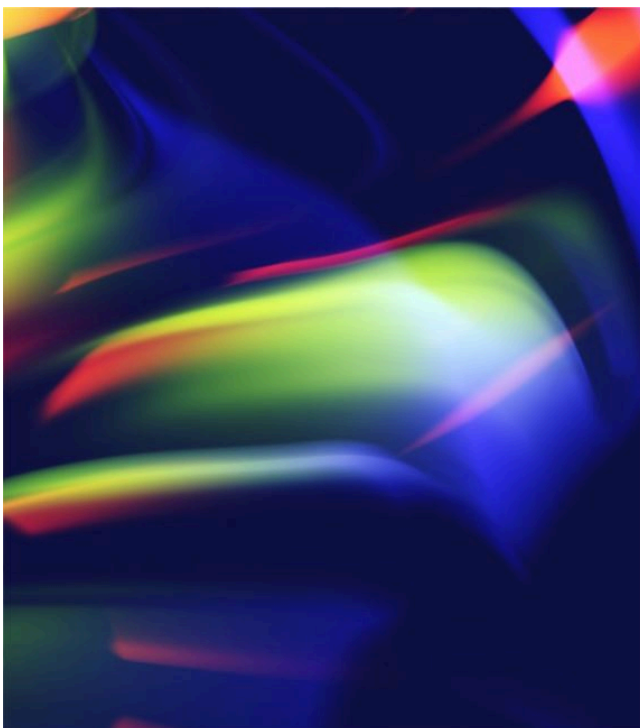
6. JavaScript moderno

Presentación

JavaScript es un lenguaje que ha sido mal entendido durante mucho tiempo. Existe una gran cantidad de tutoriales en Internet que hacen referencia a prácticas obsoletas o directamente equivocadas. Gran parte de los errores están relacionados con el ámbito de las variables, el uso de **this** o la confusión entre herencia clásica y herencia de prototipos.

El carácter dinámico de JavaScript hace que sea extremadamente **permisivo** en algunas de las operaciones relacionadas con esos aspectos. Por eso se ha desarrollado el **modo estricto**: un modo de funcionamiento que impide la ejecución de **operaciones ambiguas** y las trata como **errores explícitos**.

JavaScript además ofrece opciones muy variadas para trabajar con **objetos**. La elección de una **sintaxis** tradicional (funciones constructoras, objetos enlazados) o una basada en **class** vendrá marcada por el código existente o nuestra preferencia personal, pero es recomendable mantener la **coherencia** y utilizar la misma opción en todo el código de la aplicación.



6.1. Modo estricto

El modo estricto es una característica que aparece en el estándar EcmaScript5, aparecido en 2011. Su principal función es la de hacer que determinadas acciones que provocaban **errores silenciosos** o **efectos no deseados** se conviertan en **errores explícitos** a la hora de ejecutar los programas. El modo estricto se puede activar:

- A nivel de **script**, indicándolo al principio del archivo.
- A nivel de **función**, indicándolo en la primera línea del cuerpo de la función.
- En los archivos de **módulo**, donde está **activado por defecto**.

Importante: No es buena idea mezclar código en modo normal y modo estricto. Por ello, es más recomendable utilizar el modo estricto a nivel de *script*.

Para activar el modo estricto, hay que escribir el texto `"use strict";` o `'use strict';` en la **primera línea** del archivo de *script* o función.

Importante: El modo estricto está **activado por defecto** en los **módulos**, por lo que no hay que hacer nada para activarlo.

Una de las ventajas más interesantes del modo estricto es que previene la creación de variables accidentales en el objeto global. Consideremos un programa en el que cometemos un error al escribir un nombre de variable:

```
let miVariable = 1;

// ...Código del programa...

// Nombre de variable mal escrito (la 'v' es minúscula)
mivARIABLE = 25; // No se genera error; además, se crea una variable nueva
```

En modo normal **JavaScript no genera ningún error**, sino que directamente crea una nueva variable con el nombre **mivARIABLE** en el objeto global. Ello puede hacer que el código falle y que sea muy difícil de depurar, ya que no hay ningún error que nos dé ninguna pista. Si utilizamos modo estricto, sí que se genera error:

```
"use strict";

let miVariable = 1;

// ...Código del programa...
```

```
// Nombre de variable mal escrito

mivariable = 25; // ReferenceError: mivariable is not defined
```

Este tipo de errores suele ser muy habitual al omitir la referencia a **this** en funciones que se utilizan como constructoras de objetos. Por ejemplo, consideremos la siguiente función constructora:

```
function Coche(a, b) {
    this.marca = a;
    this.modelo = b;
}

let c1 = new Coche("Seat", "Ibiza");

console.log(c1.marca); // Seat
console.log(c1.modelo); // Ibiza
```

Si se omite el uso de **this**, quedaría de la siguiente manera:

```
function Coche(a, b) {
    marca = a; // Se crea una variable global con nombre 'marca'
    modelo = b; // Se crea una variable global con nombre 'modelo'
}

let c1 = new Coche("Seat", "Ibiza"); // No se genera ningún error

// Sin embargo...
console.log(c1.marca); // undefined
console.log(c1.modelo); // undefined

// Y además se crean dos variables globales
console.log(marca); // Seat
console.log(modelo); // Ibiza
```

Existen muchos otros casos en los que JavaScript no genera errores explícitos. Por ello, **el uso del modo estricto es recomendable**, sobre todo, para programadores con poca experiencia con el lenguaje.

6.2. ¿Clases u objetos?

Tal como hemos visto en el apartado correspondiente, JavaScript ofrece diferentes maneras de trabajar con objetos, entre las que podemos destacar el uso de **funciones constructoras**, objetos con `Object.create` o las **clases**. La elección de una opción u otra dependerá de nuestras preferencias en el caso de crear programas nuevos o de la técnica utilizada en el caso de modificar programas existentes. Lo que sí es recomendable es **mantener la coherencia y utilizar la misma técnica** en todo el código de la aplicación.

Dicho esto, en ocasiones nos encontraremos con que algunas API o librerías nos *fuerzan* a utilizar un método concreto: es el caso de la tecnología **Web Components**, un estándar en desarrollo para crear componentes HTML que puedan ser reutilizados en las páginas. Esta API del navegador utiliza la sintaxis de **clases** en su documentación. A continuación, puedes ver un ejemplo:

```
customElements.define('mi-componente',  
  class extends HTMLElement {  
    constructor() {  
      super();  
      // ...  
    }  
  }  
);
```


6.3. Manejo de errores

El manejo de errores en JavaScript se realiza con los bloques `try...catch`. Estos bloques funcionan de manera parecida a otros lenguajes. Los errores que se capturan son los que aparecen **en tiempo de ejecución**. La función `catch` define un parámetro, que es el error capturado. Dicho parámetro hace referencia a un objeto **Error**, que tiene **tres propiedades**:

- **name**. El nombre del error.
- **message**. El mensaje de error.
- **stack**. Información sobre la ruta de ejecución que ha provocado el error.

A continuación, se muestra un ejemplo:

```
try {  
    // Acceso a una propiedad de una variable que no existe  
    console.log(hola.noexiste);  
} catch (error) {  
    // El parámetro puede tener cualquier nombre  
    console.log(error.name);    // ReferenceError  
    console.log(error.message); // hola is not defined  
}
```

También es posible generar **errores personalizados** mediante `throw`:

```
try {  
    let a = "valornovalido";  
  
    if (a == "valornovalido") {  
        // Generamos un error con un mensaje personalizado  
        throw new Error("a no puede tener ese valor");  
    }  
} catch (err) {  
    console.log(err.name);    // Error  
    console.log(err.message); // a no puede tener ese valor  
}
```

Importante: `try...catch` funciona de manera síncrona. Por tanto, no funciona correctamente con funciones asíncronas (que veremos en otra unidad posterior).

En ocasiones, puede ser interesante ejecutar un trozo de código **independientemente de que se haya generado o no un error**. Consideremos, por ejemplo, una petición a un recurso remoto cuya conexión hay que cerrar, tanto si se realiza con éxito como si no. En ese caso podemos utilizar el bloque **finally**:

```
try {  
    // Apertura de conexión  
    // Procesamiento de datos  
}  
} catch( err ) {  
    // Captura de errores  
}  
} finally {  
    // Cierre de conexión  
    // Este bloque se ejecuta siempre, tanto si hay error como si no  
}
```

6.4. Evolución y futuro

JavaScript es un lenguaje en continua evolución. Goza de una enorme popularidad y además está estrechamente ligado a la web, hecho que potencia también su desarrollo. Algunas de las muchas características que se han añadido en los últimos años son:

- Operador *rest* y sintaxis *spread*.
- Promesas y funciones asíncronas.
- Generadores.
- *Proxies*.
- El sistema de módulos.
- Tipo de datos **BigInt**.
- Nuevas estructuras y sintaxis.
- Mejoras de sintaxis para la simplificación de estructuras de código.

Algunas de ellas las hemos estudiado en esta unidad. Otras las veremos en unidades posteriores (como las promesas y las funciones asíncronas), mientras que el resto (generadores, *proxies*) no las estudiaremos aquí por motivos de espacio y complejidad.

En paralelo a la evolución del lenguaje, se produce también una enorme evolución en las **API de los navegadores**, que continuamente están desarrollándose y ampliando funcionalidades. En la siguiente unidad estudiaremos algunas de ellas y veremos cómo se benefician también de la evolución del lenguaje.

Por ambos motivos, podemos esperar que JavaScript seguirá evolucionando en los próximos años, incorporando nuevas características que permitirán a los desarrolladores crear aplicaciones cada vez más complejas y potentes. Como futuro desarrollador, tienes el reto de seguir formándote a lo largo de tu carrera profesional para actualizar tus conocimientos y mantenerte al día.

Test 6

6. JavaScript moderno

1. El modo estricto es el modo por defecto en los módulos.

- Verdadero
- Falso

2. El modo estricto...

- Impide la creación de variables globales accidentales sin declarar.
- Prohíbe el uso de `var`.
- Cambia el ámbito de todas las variables a ámbito de bloque.
- Impide la creación de cualquier variable global, aunque esté declarada.

3. La sintaxis `class...`

- Es la única válida en modo estricto.
- Es mejor que la sintaxis basada en funciones constructoras.
- Es utilizada en algunas API, como Web Components.
- No utiliza herencia de prototipos.

4. ¿Qué palabra reservada se utiliza para lanzar un error personalizado?

- `try`.
- `throw`.
- `catch`.
- `Error`.

5. `try...catch` funciona tanto para código síncrono como asíncrono.

- Verdadero
 - Falso
-

Actividad 2.11

Comprobación del modo estricto

Ejecuta el siguiente programa en una etiqueta `<script>` dentro de una página HTML de las siguientes tres maneras:

- De manera convencional.
- Activando el modo estricto.
- Sin modo estricto, pero activando el atributo `type="module"` en la etiqueta `<script>`.

Ejecuta el siguiente programa, con y sin modo estricto. ¿Sabrías explicar por qué no funciona en modo estricto?

```
function Enemigo(s, f) {  
    salud = s;  
    fuerza = f;  
}  
  
Enemigo.prototype.atacar = function() {  
    console.log(`Atacando con fuerza ${fuerza}`);  
}  
  
let enemigo1 = new Enemigo(50, 20);  
let enemigo2 = new Enemigo(40, 15);  
  
enemigo1.atacar();  
enemigo2.atacar();
```

En cada uno de los tres casos, responde a las siguientes preguntas:

- ¿El programa se ejecuta sin fallos?
 - En caso afirmativo, ¿funciona correctamente de acuerdo con lo que se espera?
 - ¿Puedes identificar los errores y explicar por qué se producen?
-

Actividad 2.12

Parámetros de las funciones

La siguiente función espera recibir como parámetro un objeto con una propiedad denominada **data**. Sin embargo, puedes comprobar que si se llama a la función sin parámetros o con **null** se producirá un error. Modifica el código para que se capture dicho error y se muestre un mensaje en la consola con el que se indique el problema. En el caso de que produzca un error, la función deberá devolver **-1**.

```
function procesar(mensaje) {  
    return mensaje.data;  
}
```

Claves de resolución. El acceso a **mensaje.data** provoca un error de acceso si **mensaje** es **null** o **undefined**. Por tanto, hay que englobar el código de la función en un bloque de control de errores para capturarlo. El bloque de procesamiento del error deberá realizar dos tareas: escribir el mensaje en la consola y devolver el valor **-1** como resultado de la función.
